



For Instructor Use Only.  
This document should not be distributed.

# Oracle AI Vector Search Fundamentals

Activity Guide  
D1109439GC10



**Copyright © 2025, Oracle and/or its affiliates.**

## **Disclaimer**

This document contains proprietary information and is protected by copyright and other intellectual property laws. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

## **Restricted Rights Notice**

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

## **Trademark Notice**

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

## **Third-Party Content, Products, and Services Disclaimer**

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1001242025

For Instructor Use Only.  
This document should not be distributed.

## Table of Contents

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| Course Practice Environment: Security Credentials .....                      | 5          |
| <b>Practices for Lesson 1: Course Overview .....</b>                         | <b>7</b>   |
| Practice 1-1: Configuring the Landing Pad VM.....                            | 8          |
| Practice 1-2: Updating the <code>tnsnames.ora</code> File .....              | 10         |
| Practice 1-3: Creating a User.....                                           | 11         |
| <b>Practices for Lesson 2: Why use Oracle AI Vector Search? .....</b>        | <b>13</b>  |
| Practices for Lesson 2 .....                                                 | 14         |
| <b>Practices for Lesson 3: Oracle AI Vector Search Workflow .....</b>        | <b>15</b>  |
| Practices for Lesson 3 .....                                                 | 16         |
| <b>Practices for Lesson 4: Running Basic Queries on Vectors .....</b>        | <b>17</b>  |
| Practice 4-1: Selecting the Values from a Vector .....                       | 18         |
| <b>Practices for Lesson 5: Vector Indexes and Memory .....</b>               | <b>21</b>  |
| Practices for Lesson 5 .....                                                 | 22         |
| <b>Practices for Lesson 6: DML Operations on Vectors .....</b>               | <b>23</b>  |
| Practice 6-1: Performing DML Operations on Vectors .....                     | 24         |
| <b>Practices for Lesson 7: Vector DDL .....</b>                              | <b>31</b>  |
| Practice 7-1: Performing DDL Operations on Vector Columns .....              | 32         |
| <b>Practices for Lesson 8: Creating and Finding the Nearest Vectors.....</b> | <b>37</b>  |
| Practice 8-1: Vector Distance .....                                          | 38         |
| <b>Practices for Lesson 9: Finding the Closest Vectors .....</b>             | <b>47</b>  |
| Practice 9-1: Finding the Closest Vectors to a Given Vector.....             | 48         |
| Practice 9-2: Finding Vectors Based on Vectors Clustered in Groups .....     | 61         |
| <b>Practices for Lesson 10: Narrowing Search Results .....</b>               | <b>71</b>  |
| Practice 10-1: Setting Up a Table with Sample Vectors .....                  | 72         |
| Practice 10-2: Attribute Filtering.....                                      | 78         |
| <b>Practices for Lesson 11: Testing Other Distance Functions .....</b>       | <b>89</b>  |
| Practice 11-1: Setting Up a Table with Sample Vectors .....                  | 90         |
| Practice 11-2: Using Alternative Distance Functions.....                     | 93         |
| Practice 11-3: Using Alternative Vector Distance Functions .....             | 96         |
| Practice 11-4: Using Shorthand Syntax to Look for the Closest Vectors .....  | 99         |
| <b>Practices for Lesson 12: Testing Other Vector Functions.....</b>          | <b>103</b> |
| Practice 12-1: Other Vector Functions.....                                   | 104        |

For Instructor Use Only.  
This document should not be distributed.

## Course Practice Environment: Security Credentials

---

For operating system (Linux) usernames and passwords, see the following:

- If you are attending a classroom-based or live virtual class, ask your instructor or LVC producer for OS credential information.
- If you are using a self-study format, refer to the communication that you received from Oracle University for this course.

| Operating System Credentials |          |                            |
|------------------------------|----------|----------------------------|
| Practice Environment         | Username | Password                   |
| Gateway VM (edvmr1p0)        | root     | <i>Read the note above</i> |
|                              | oracle   |                            |

For product-specific credentials used in this course, see the following table:

| Product-Specific Credentials |           |                           |
|------------------------------|-----------|---------------------------|
| Product/Application          | Username  | Password (Case Sensitive) |
| ORCLCDB and ORCLPDB1         | All users | WE1come123##              |

For Instructor Use Only.  
This document should not be distributed.

For Instructor Use Only.  
This document should not be distributed.

**Practices for Lesson 1:  
Course Overview**

For Instructor Use Only.  
This document should not be distributed.

## Practice 1-1: Configuring the Landing Pad VM

### Overview

The lab environment consists of the following VMs:

- **Landing pad VM:** You initially connect to this VM and connect to an OCI DB System.
- **OCI DB System:** The OCI DB System VM is dedicated to Oracle Database 23ai, installed with a pre-created CDB and pre-created PDBs for your practices.

In this practice, you configure the landing pad VM to connect to the OCI DB System.

### Tasks

1. Ensure that you have collected the following information from the Lab tab of the course:
  - The Public IP address of the OCI DB System
  - The Private IP address of the OCI DB System
  - The text of the SSH private key that will allow you to connect to the OCI DB System. Be ready to copy this text into an SSH private key file that you are going to create. The text starts with `-----BEGIN RSA PRIVATE KEY-----` and ends with `-----END RSA PRIVATE KEY-----`. Both lines will be included in the SSH private key file.

2. Review the usage of the aliases.

| Alias                   | Note                                                                 |
|-------------------------|----------------------------------------------------------------------|
| <code>ssh-oracle</code> | Used to connect to the OCI DB system as the <code>oracle</code> user |
| <code>ssh-sqldev</code> | Used to create a SSH tunnel for SQL Developer connection             |

3. Open a terminal window and update each alias in the `.bashrc` file using a text editor.
  - Replace `<OCI Public IP>` with the Public IP obtained in step 1.
  - Replace `<OCI Private IP>` with the Private IP obtained in step 1.

```
[oracle@edvmrlp0 ~]$ vi .bashrc
alias ssh-oracle="ssh -i /home/oracle/.ssh/mykey oracle@<OCI
Public IP>"
alias ssh-sqldev="ssh -i /home/oracle/.ssh/mykey -N -L
localhost:1521:<OCI Private IP>:1521 oracle@<OCI Public IP> &"

:wq!
[oracle@edvmrlp0 ~]$
```

4. Apply the changes.

```
[oracle@edvmrlp0 ~]$ . .bashrc
[oracle@edvmrlp0 ~]$
```



5. Copy and paste the SSH private key obtained in step 1 into the `mykey` file in the `/home/oracle/.ssh` directory.

```
[oracle@edvmr1p0 ~]$ vi /home/oracle/.ssh/mykey
-----BEGIN RSA PRIVATE KEY-----

<SSH Private Key Truncated>

-----END RSA PRIVATE KEY-----

:wq!
[oracle@edvmr1p0 ~]$
```

6. Close all terminal windows.

## Practice 1-2: Updating the `tnsnames.ora` File

---

### Overview

In this practice, you update the `tnsnames.ora` file in the OCI DB system.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmr1p0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Add the alias named `ORCLPDB1` to the `tnsnames.ora` file.

```
[oracle@host01 ~]$ vi $ORACLE_HOME/network/admin/tnsnames.ora  
...  
  
ORCLPDB1 =  
  (DESCRIPTION =  
    (ADDRESS = (PROTOCOL = TCP) (HOST =  
host01.public.myvcn.oraclevcn.com) (PORT = 1521))  
    (CONNECT_DATA =  
      (SERVER = DEDICATED)  
      (SERVICE_NAME = orclpdb1.public.myvcn.oraclevcn.com)  
    )  
  )  
  
:wq!  
[oracle@host01 ~]$
```

3. Close the terminal window.

For Instructor Use Only.  
This document should not be distributed.

## Practice 1-3: Creating a User

---

### Overview

In this practice you create a user called `vector` and grant the `DB_DEVELOPER_ROLE` role to the vector user.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Create the `vector` user in `ORCLPDB1` and grant the `DB_DEVELOPER_ROLE`.

```
[oracle@host01 ~]$ sqlplus / as sysdba  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Wed May 29 20:19:55  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Connected to:  
Oracle Database 23ai Standard Edition 2 Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL> alter session set container=orclpdb1;  
  
Session altered.  
  
SQL> create user vector identified by WELCOME123##  
      default tablespace users  
      quota 50M on users;  
  
User created.  
  
SQL> grant DB_DEVELOPER_ROLE to vector;  
  
Grant succeeded.  
  
[oracle@host01 ~]$
```

3. Close the terminal window.

For Instructor Use Only.  
This document should not be distributed.

For Instructor Use Only.  
This document should not be distributed.

## **Practices for Lesson 2: Why use Oracle AI Vector Search?**

For Instructor Use Only.  
This document should not be distributed.

## Practices for Lesson 2

---

There are no practices for this lesson.

For Instructor Use Only.  
This document should not be distributed.



## **Practices for Lesson 3: Oracle AI Vector Search Workflow**

For Instructor Use Only.  
This document should not be distributed.

## Practices for Lesson 3

---

There are no practices for this lesson.

For Instructor Use Only.  
This document should not be distributed.

**Practices for Lesson 4:  
Running Basic Queries on  
Vectors**

For Instructor Use Only.  
This document should not be distributed.

## Practice 4-1: Selecting the Values from a Vector

---

### Overview

In this practice, you will perform the following vector operations:

- Create a table with a single vector column.
- Insert a few rows.
- Select the values from a vector.

### Assumptions

You have completed Practices 1-1, 1-2, and 1-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01 ~]$ sqlplus vector/WELCOME123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Create a vector in a table. **Note:** You will notice when we describe the table, we can see the new VECTOR data type is displayed.

```
SQL> DROP TABLE IF EXISTS t1;

Table dropped.

SQL> CREATE TABLE IF NOT EXISTS t1 (v vector);

Table created.

SQL> DESC t1
Name                               Null?      Type
-----
v                                   NULL       VECTOR(*, *)

SQL>
```

4. Insert some trivial vectors into the vector table. **Note:** *Vectors are an array of numbers and can be null. However individual vector array elements numbers cannot be null.*

- You need to separate each number with a comma.
- You need to enclose the array with single quotes.

```
SQL> INSERT INTO t1 VALUES ('[1.1, 2.7, 3.141592653589793238]'),
                             ('[9.34, 0.0, -6.923]'),
                             ('[-2.01, 5, -25.8]'),
                             ('[-8.2, -5, -1013.6]'),
                             ('[7.3]'),
                             ('[2.9]'),
                             ('[1, 2, 3, 4, 5]');

7 rows created.

SQL>
```

5. Now let's **SELECT** the values of a vector. **Note:** You will notice that the number is displayed in scientific format.

```
SQL> SELECT * FROM t1;

V
-----
[1.10000002E+000,2.70000005E+000,3.14159274E+000]
[9.34000015E+000,0,-6.92299986E+000]
[-2.00999999E+000,5.0E+000,-2.57999992E+001]
[-8.19999981E+000,-5.0E+000,-1.01359998E+003]
[7.30000019E+000]
[2.9000001E+000]
[1.0E+000,2.0E+000,3.0E+000,4.0E+000,5.0E+000]

7 rows selected.

SQL>
```

6. Attempt to perform comparison operations between vectors. **Note:** It is not possible to perform comparison operations between vectors.

```
SQL> SELECT v FROM t1 where v = '[2,9]';
SELECT v FROM t1 where v = '[2,9]'
*

ERROR at line 1:
ORA-22848: cannot use VECTOR type as comparison key
Help: https://docs.oracle.com/error-help/db/ora-22848/

SQL> SELECT v FROM t1 where v <= '[2,9]';
SELECT v FROM t1 where v <= '[2,9]'
*

ERROR at line 1:
ORA-22848: cannot use VECTOR type as comparison key
Help: https://docs.oracle.com/error-help/db/ora-22848/

SQL>
```

7. Close the terminal window.

**Practices for Lesson 5:**  
**Vector Indexes and Memory**

For Instructor Use Only.  
This document should not be distributed.

## Practices for Lesson 5

---

There are no practices for this lesson.

For Instructor Use Only.  
This document should not be distributed.



## **Practices for Lesson 6: DML Operations on Vectors**

For Instructor Use Only.  
This document should not be distributed.

## Practice 6-1: Performing DML Operations on Vectors

---

### Overview

In this practice, you will see how regular DML (data manipulation language) operations can be used with vector columns.

### Assumptions

You have completed Practices 1-1, I-2, and I-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Create a table called T2 and perform DML operations on vectors.
  - a. Create a table with multiple columns including a vector column.

```
SQL> CREATE TABLE IF NOT EXISTS t2
      ( id          NUMBER NOT NULL,
        name        VARCHAR2(32),
        v1          VECTOR,
        PRIMARY KEY (id)
      );
```

Table created.

```
SQL> DESC t2
```

| Name | Null?    | Type         |
|------|----------|--------------|
| ID   | NOT NULL | NUMBER       |
| NAME |          | VARCHAR2(32) |
| V1   |          | VECTOR(*, *) |

```
SQL>
```

- b. Insert rows into the newly created table.

```
SQL> INSERT INTO t2 VALUES (1, 'A', '[1.1]'),
                             (2, 'B', '[2.2]'),
                             (3, 'C', '[3.3]'),
                             (4, 'D', '[4.4]'),
                             (5, 'E', '[5.5]');
```

5 rows created.

```
SQL> SELECT * FROM t2;
```

| ID | NAME | V1                |
|----|------|-------------------|
| 1  | A    | [1.10000002E+000] |
| 2  | B    | [2.20000005E+000] |
| 3  | C    | [3.29999995E+000] |
| 4  | D    | [4.4000001E+000]  |
| 5  | E    | [5.5E+000]        |

```
SQL>
```

- c. Update a vector based upon another column in the table.

```
SQL> UPDATE t2
      SET   v1 = '[2.9]'
      WHERE id = 2;
```

1 row updated.

```
SQL> SELECT * FROM t2
      WHERE id = 2;
```

| ID | NAME | V1               |
|----|------|------------------|
| 2  | B    | [2.9000001E+000] |

SQL>

- d. Delete rows containing vectors in the table.

```
SQL> DELETE FROM t2
      WHERE id IN (1, 3);
```

2 rows deleted.

```
SQL> SELECT * FROM t2;
```

| ID | NAME | V1               |
|----|------|------------------|
| 2  | B    | [2.9000001E+000] |
| 4  | D    | [4.4000001E+000] |
| 5  | E    | [5.5E+000]       |

SQL>

4. Create a table called **T3** and perform DML operations on vectors.
  - a. Create a table with multiple vectors. **Note:** It is most likely that normalized tables will have one vector per table.

```
SQL> CREATE TABLE IF NOT EXISTS t3
      ( id          NUMBER NOT NULL,
        name        VARCHAR2(32),
        v1          VECTOR,
        v2          VECTOR,
        v3          VECTOR,
        PRIMARY KEY (id)
      );
```

Table created.

```
SQL> DESC t3
```

| Name | Null?    | Type         |
|------|----------|--------------|
| ID   | NOT NULL | NUMBER       |
| NAME |          | VARCHAR2(32) |
| V1   |          | VECTOR(*, *) |
| V2   |          | VECTOR(*, *) |
| V3   |          | VECTOR(*, *) |

```
SQL>
```

- b. Insert a row into the table.

```
SQL> INSERT INTO t3 VALUES
      (1,
       'One',
       '[2.3, 4.5, 0.1]',
       '[1.3]',
       '[4.981, -6.3]'
      );
```

1 row created.

```
SQL> SELECT * FROM t3;
```

| ID | NAME | V1                                                 | V2                    | V3                                         |
|----|------|----------------------------------------------------|-----------------------|--------------------------------------------|
| 1  | One  | [2.29999995E+00<br>0,4.5E+000,1.00<br>000001E-001] | [1.29999995E+00<br>0] | [4.98099995E+00<br>0,-6.30000019E+<br>000] |

```
SQL>
```

5. Create a table called T4 and perform DML operations on vectors.
- a. Create a table with a fixed number and number-format vector.

```
SQL> CREATE TABLE IF NOT EXISTS t4
      ( v    VECTOR(3, FLOAT32) );
```

Table created.

```
SQL> DESC t4
```

| Name | Null? | Type               |
|------|-------|--------------------|
| V    |       | VECTOR(3, FLOAT32) |

```
SQL>
```

- b. See how vector formats are implicitly controlled once they are defined.

In this example we will see:

- The wrong number of elements will be rejected.
- The wrong format for the elements will be rejected.

**Note:** Observe the error messages thrown for the last three insert statements.

```
SQL> INSERT INTO t4 VALUES ('[1.1, 2.2, 3.3]');
```

1 row created.

```
SQL> INSERT INTO t4 VALUES ('[1.2, 2.3, 3.4]');
```

1 row created.

```
SQL> INSERT INTO t4 VALUES ('[1.2, 2.3, 3.4]');
```

1 row created.

```
SQL> INSERT INTO t4 VALUES ('[1.3]');
```

```
INSERT INTO t4 VALUES ('[1.3]')
```

\*

ERROR at line 1:

ORA-51803: Vector dimension count must match the dimension count specified in the column definition (actual: 1, required: 3).

Help: <https://docs.oracle.com/error-help/db/ora-51803/>

```
SQL> INSERT INTO t4 VALUES ('[1.3, 2.4, 3.5, 4.1]');
INSERT INTO t4 VALUES ('[1.3, 2.4, 3.5, 4.1]')
      *
```

ERROR at line 1:  
ORA-51803: Vector dimension count must match the dimension count specified in the column definition (actual: 4, required: 3).  
Help: <https://docs.oracle.com/error-help/db/ora-51803/>

```
SQL> INSERT INTO t4 VALUES ('[1.4, 2.5, a]');
INSERT INTO t4 VALUES ('[1.4, 2.5, a]')
      *
```

ERROR at line 1:  
ORA-51805: Vector is not properly formatted (dimension value 3 is either not a number or infinity).  
Help: <https://docs.oracle.com/error-help/db/ora-51805/>

```
SQL>
```

6. Close the terminal window.

For Instructor Use Only.  
This document should not be distributed.



## **Practices for Lesson 7: Vector DDL**

For Instructor Use Only.  
This document should not be distributed.

## Practice 7-1: Performing DDL Operations on Vector Columns

---

### Overview

In this practice, you will see how you can perform the various vector DDL (Data Definition Language) operations.

### Assumptions

You have completed Practices 1-1, 1-2, and 1-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmr1p0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

Last Successful login time: Tue May 14 2024 20:46:30 +00:00

Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

SQL>
```

3. Create tables called `T5` with different vector formats.
  - a. Create a table with the following vector formats:
    - `vector(*, float64)` means any number of dimensions with `float64` format.
    - `vector(5, *)` means five dimensions with any number format.
    - `vector(*, *)` means any number of dimensions with any number format.
    - `vector` means any number of dimensions with `float32` format.
    - `vector(1, int8)` means one dimension with `int8` format.
    - `int8` is a single byte.

For Instructor Use Only.  
This document should not be distributed.

```
SQL> CREATE TABLE IF NOT EXISTS t5
      ( v1      VECTOR(3, float32),
        v2      VECTOR(2, float64),
        v3      VECTOR(1, int8),
        v4      VECTOR(1, *),
        v5      VECTOR(*, float32),
        v6      VECTOR(*, *),
        v7      VECTOR
      );
```

Table created.

```
SQL> DESC t5
```

| Name | Null? | Type               |
|------|-------|--------------------|
| V1   |       | VECTOR(3, FLOAT32) |
| V2   |       | VECTOR(2, FLOAT64) |
| V3   |       | VECTOR(1, INT8)    |
| V4   |       | VECTOR(1, *)       |
| V5   |       | VECTOR(*, FLOAT32) |
| V6   |       | VECTOR(*, *)       |
| V7   |       | VECTOR(*, *)       |

```
SQL>
```

- b. Now, let's insert data into the vectors.

```
SQL> INSERT INTO t5 VALUES ('[1.1, 2.2, 3.3]',
                             '[1.1, 2.2]',
                             '[7]',
                             '[9]',
                             '[1.1, 2.2, 3.3, 4.4, 5.5]',
                             '[1.1, 2.2]',
                             '[1.1, 2.2, 3.3, 4.4, 5.5, 6.6]'
                           );
```

1 row created.

```
SQL> SELECT * FROM t5;

V1
-----
V2
-----
V3
-----
V4
-----
V5
-----
V6
-----
V7
-----

[1.10000002E+000,2.20000005E+000,3.29999995E+000]
[1.1000000000000001E+000,2.2000000000000002E+000]
[7]
[9.0E+000]
[1.10000002E+000,2.20000005E+000,3.29999995E+000,4.4000001E+000,5.5E+000]
[1.10000002E+000,2.20000005E+000]
[1.10000002E+000,2.20000005E+000,3.29999995E+000,4.4000001E+000,5.5E+000,6.59999
99E+000]

1 row selected.

SQL>
```

4. Perform regular DDL on a table with vectors.
  - a. Add a vector to an existing table. **Note:** Regular DDL operations are permitted on tables with vectors.

```
SQL> CREATE TABLE IF NOT EXISTS t6
      (
        id          NUMBER NOT NULL,
        name        VARCHAR2(32),
                  PRIMARY KEY (id)
      );

Table created.

SQL> ALTER TABLE t6 ADD v1 VECTOR;

Table altered.

SQL> ALTER TABLE t6 ADD v2 VECTOR(2, float32);

Table altered.
```

```
SQL> DESC t6
```

| Name | Null?    | Type               |
|------|----------|--------------------|
| ID   | NOT NULL | NUMBER             |
| NAME |          | VARCHAR2(32)       |
| V1   |          | VECTOR(*, *)       |
| V2   |          | VECTOR(2, FLOAT32) |

```
SQL>
```

- b. Remove a vector from a table or even drop an entire table containing a vector.

```
SQL> ALTER TABLE t6 DROP COLUMN v2;
```

Table altered.

```
SQL> DESC t6
```

| Name | Null?    | Type         |
|------|----------|--------------|
| ID   | NOT NULL | NUMBER       |
| NAME |          | VARCHAR2(32) |
| V1   |          | VECTOR(*, *) |

```
SQL> DROP TABLE IF EXISTS t6;
```

Table dropped.

```
SQL>
```

- c. Create a table with a vector from another table with a vector. **Note:** In this example, we are using a CTAS (Create Table As Select) operation.

```
SQL> CREATE TABLE IF NOT EXISTS t7  
      AS SELECT * FROM t3;
```

Table created.

```
SQL> DESC t7
```

| Name | Null?    | Type         |
|------|----------|--------------|
| ID   | NOT NULL | NUMBER       |
| NAME |          | VARCHAR2(32) |
| V1   |          | VECTOR(*, *) |
| V2   |          | VECTOR(*, *) |
| V3   |          | VECTOR(*, *) |

```
SQL> SELECT * FROM t7;
```

| ID | NAME | V1                                                    | V2              | V3                                         |
|----|------|-------------------------------------------------------|-----------------|--------------------------------------------|
| 1  | One  | [2.29999995E+00<br>0,4.5E+000,1.00 0]<br>000001E-001] | [1.29999995E+00 | [4.98099995E+00<br>0,-6.30000019E+<br>000] |

```
SQL>
```

5. Close the terminal window.

**Practices for Lesson 8:  
Creating and Finding the  
Nearest Vectors**

For Instructor Use Only.  
This document should not be distributed.

## Practice 8-1: Vector Distance

---

### Overview

In this practice, you will perform the following tasks:

- Use the SQL `vector()` constructor to create vectors.
- Use the `vector_distance()` function to return the distance between two vectors.

`vector_distance()` is the most important vector SQL function as it enables similarity search. The `vector_distance()` SQL function returns the distance between two vectors

### Assumptions

You have completed Practices 1-1, 1-2, and 1-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmr1p0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to `ORCLPDB1` as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WELCOME123##@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

Last Successful login time: Tue May 14 2024 20:46:30 +00:00

Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

SQL>
```

For Instructor Use Only.  
This document should not be distributed.



3. Use the SQL `vector()` constructor to create vectors.

- a. Run the following commands to echo your SQL command and format output result.

```
SQL> SET ECHO ON
SQL> SET FEEDBACK 1
SQL> SET NUMWIDTH 10
SQL> SET LINESIZE 80
SQL> SET TRIMSPOOL ON
SQL> SET TAB OFF
SQL> SET PAGESIZE 100
SQL>
```

- b. Run the following queries using the `vector()` constructor.

**Note:** You can create vectors without inserting them into a vector column. This is useful for learning how to use vectors with a small number of dimensions. Vectors created via Embedding Models from companies like OpenAI or Cohere can have thousands of dimensions.

You need to specify the vector values, with optional number of dimensions and format. The following constructors represent the above two-dimensional coordinates. In Oracle 23ai, you do not need to specify a table for a SQL function. The values in the vector will be in scientific format.

```
SQL> SELECT VECTOR(' [0,0] ');

VECTOR(' [0,0] ')
-----
[0,0]

1 row selected.

SQL> SELECT VECTOR(' [10,0] ');

VECTOR(' [10,0] ')
-----
[1.0E+001,0]

1 row selected.
```

```
SQL> SELECT VECTOR('[0,5]', 2, float32);
```

```
VECTOR('[0,5]',2,FLOAT32)
```

```
-----  
[0,5.0E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR('[4,3]');
```

```
VECTOR('[4,3]')
```

```
-----  
[4.0E+000,3.0E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR('[5,-2]');
```

```
VECTOR('[5,-2]')
```

```
-----  
[5.0E+000,-2.0E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR('[-3,-4]');
```

```
VECTOR('[-3,-4]')
```

```
-----  
[-3.0E+000,-4.0E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR('[3.14,2.718]');
```

```
VECTOR('[3.14,2.718]')
```

```
-----  
[3.1400001E+000,2.71799994E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR(' [-5.3,4.2] ');
```

```
VECTOR(' [-5.3,4.2] ')
```

```
-----  
[-5.30000019E+000,4.19999981E+000]
```

```
1 row selected.
```

```
SQL> SELECT VECTOR(' [-7,-9] ');
```

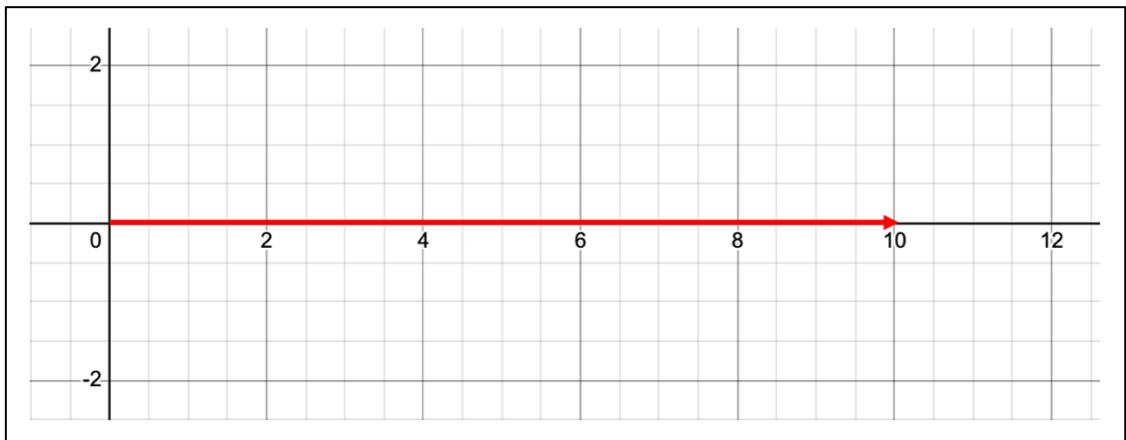
```
VECTOR(' [-7,-9] ')
```

```
-----  
[-7.0E+000,-9.0E+000]
```

```
1 row selected.
```

```
SQL>
```

4. Use the `vector_distance` SQL function to get the horizontal distance.
  - a. The `vector_distance()` SQL function returns the distance between two vectors. For example, the distance between (0,0) and (10,0) is 10. It is just the horizontal offset of 10 as the vertical offset is zero.



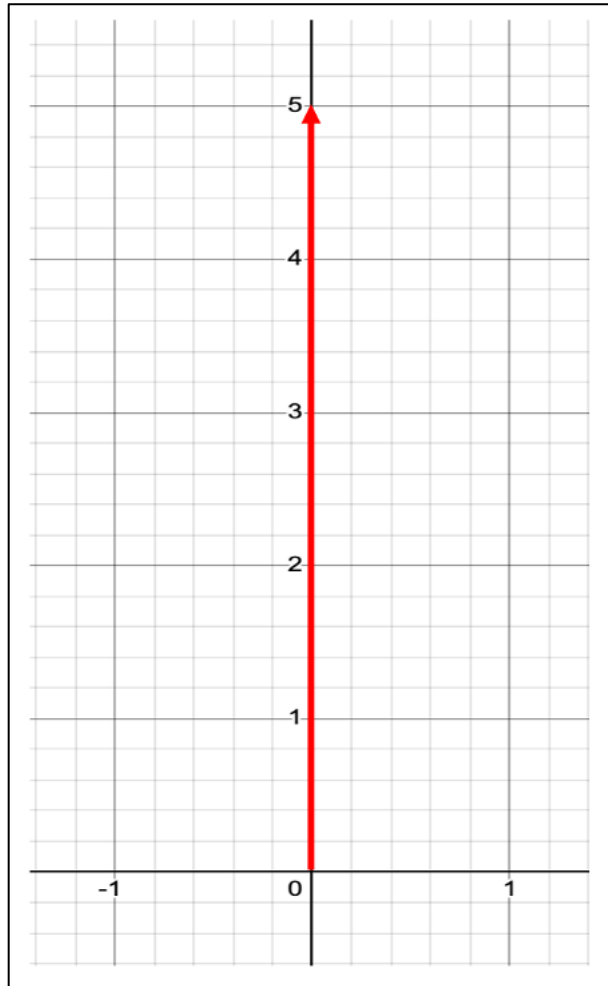
- b. Use the `vector_distance()` SQL function to return the distance between two vectors.

```
SQL> SELECT VECTOR_DISTANCE(  
        VECTOR(' [0, 0] '),  
        VECTOR(' [10, 0] '),  
        EUCLIDEAN) DISTANCE;  
  
DISTANCE  
-----  
1.0E+001  
  
1 row selected.
```

- c. Use the `to_number()` function to improve readability.

```
SQL> SELECT TO_NUMBER(VECTOR_DISTANCE(  
        VECTOR(' [0, 0] '),  
        VECTOR(' [10, 0] '),  
        EUCLIDEAN)) DISTANCE;  
  
DISTANCE  
-----  
10  
  
1 row selected.  
  
SQL>
```

5. Use the `vector_distance` SQL function to get the vertical distance.
- a. The distance between (0,0) and (0,5) is 5. It is just the vertical offset of 5 as the horizontal offset is zero.



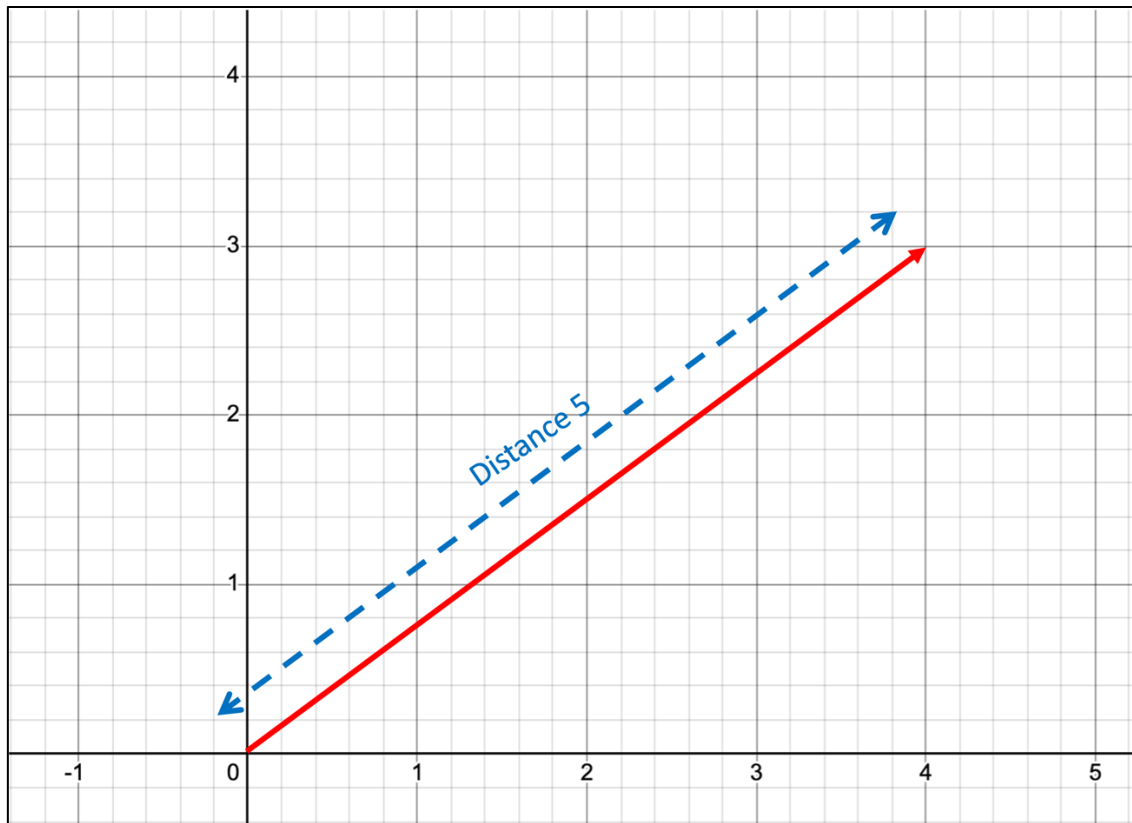
- b. Use the `vector_distance()` SQL function to return the distance between two vectors.

```
SQL> SELECT TO_NUMBER(VECTOR_DISTANCE(  
    VECTOR(' [0, 0] ', 2, FLOAT32),  
    VECTOR(' [0, 5] ', 2, FLOAT32),  
    EUCLIDEAN)) DISTANCE;
```

```
    DISTANCE  
-----  
          5  
  
1 row selected.
```

```
SQL>
```

6. Use the `vector_distance` SQL function to get the length of the hypotenuse of a triangle.
- a. The distance between (0,0) and (4,3) is 5. This is the length of the hypotenuse of a triangle. This is a 3,4,5 triangle so the length must be 5. Verify by using the Pythagorean Theorem  $a^2 + b^2 = c^2$ .



- b. Use the `vector_distance()` SQL function to return the distance between two vectors.

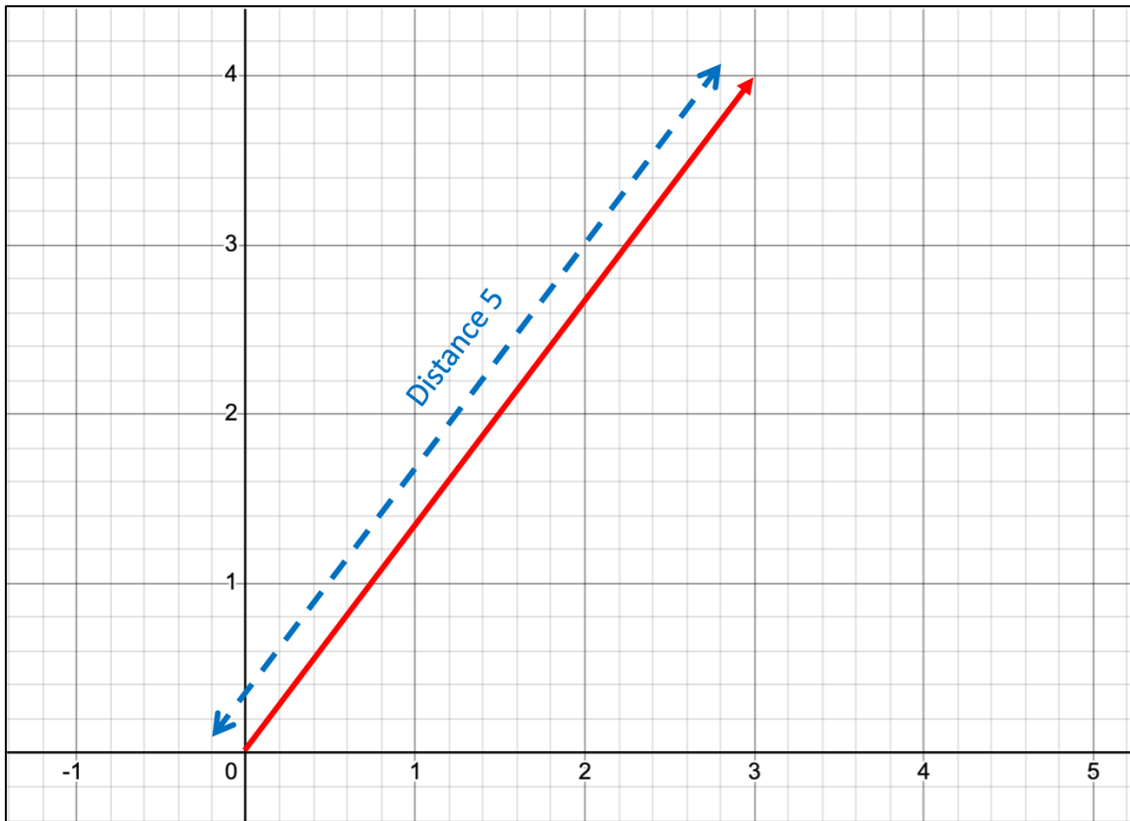
```
SQL> SELECT TO_NUMBER(VECTOR_DISTANCE(  
      VECTOR(' [0, 0] ', 2, FLOAT32),  
      VECTOR(' [4, 3] ', 2, FLOAT32),  
      EUCLIDEAN)) DISTANCE;
```

```
      DISTANCE  
-----  
              5
```

```
1 row selected.
```

```
SQL>
```

7. Use the `vector_distance` SQL function to get the length of the hypotenuse of a triangle.
- a. The distance between (0,0) and (3,4) is 5. This is also a 3,4,5 triangle so the length must be 5.



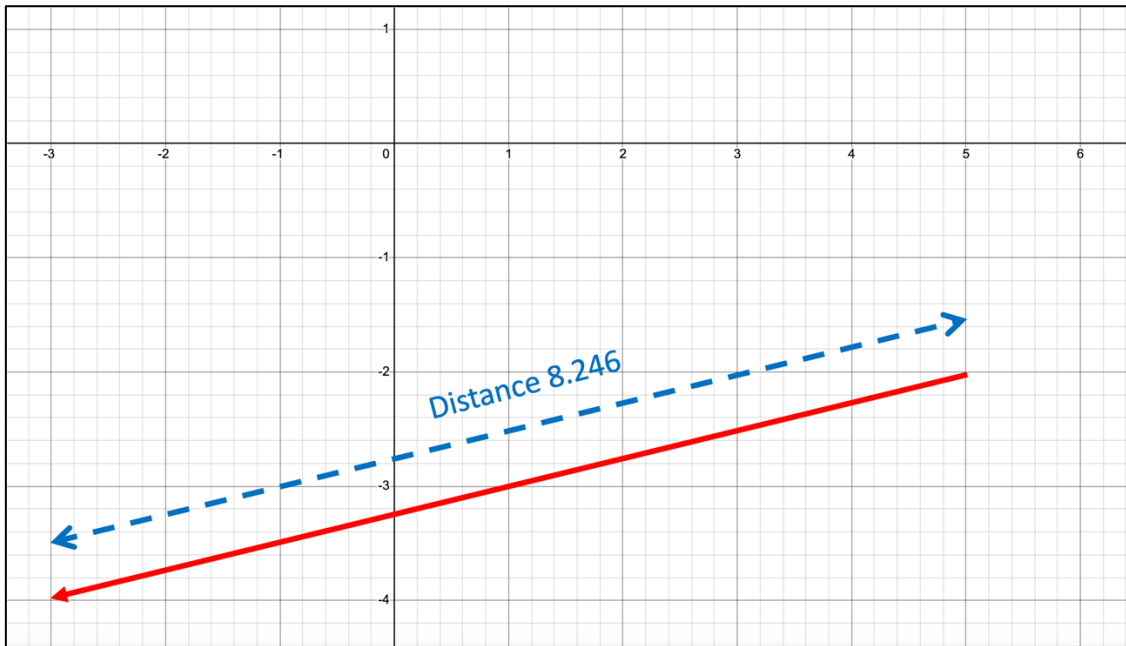
- b. Use the `vector_distance()` SQL function to return the distance between two vectors.

```
SQL> SELECT TO_NUMBER(VECTOR_DISTANCE(  
      VECTOR(' [0, 0] '),  
      VECTOR(' [3, 4] '),  
      EUCLIDEAN)) DISTANCE;
```

```
      DISTANCE  
-----  
              5  
  
1 row selected.
```

```
SQL>
```

8. Use the `vector_distance` SQL function to get the length of the hypotenuse of a triangle.
- a. You do not need to use the origin (0,0) for the distance. The distance between (5,-2) and (-3,-4) is 8.246. Verify by using the Pythagorean Theorem ( $a^2 + b^2 = c^2$ ).



- b. Use the `vector_distance()` SQL function to return the distance between two vectors.

```
SQL> SELECT TO_NUMBER(VECTOR_DISTANCE(  
      VECTOR(' [3.14, 2.718] '),  
      VECTOR(' [-5.3, 4.2] '),  
      EUCLIDEAN)) DISTANCE;
```

```
      DISTANCE  
-----  
8.56912708  
  
1 row selected.
```

```
SQL>
```

9. Close the terminal window.



**Practices for Lesson 9:  
Finding the Closest Vectors**

For Instructor Use Only.  
This document should not be distributed.

## Practice 9-1: Finding the Closest Vectors to a Given Vector

---

### Overview

In this practice, you will perform the following tasks:

- Set up a table with vectors to be used in the practice.
- Look for the closest vectors to a given vector.

### Assumptions

You have completed Practices 1-1, 1-2, and 1-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrip0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

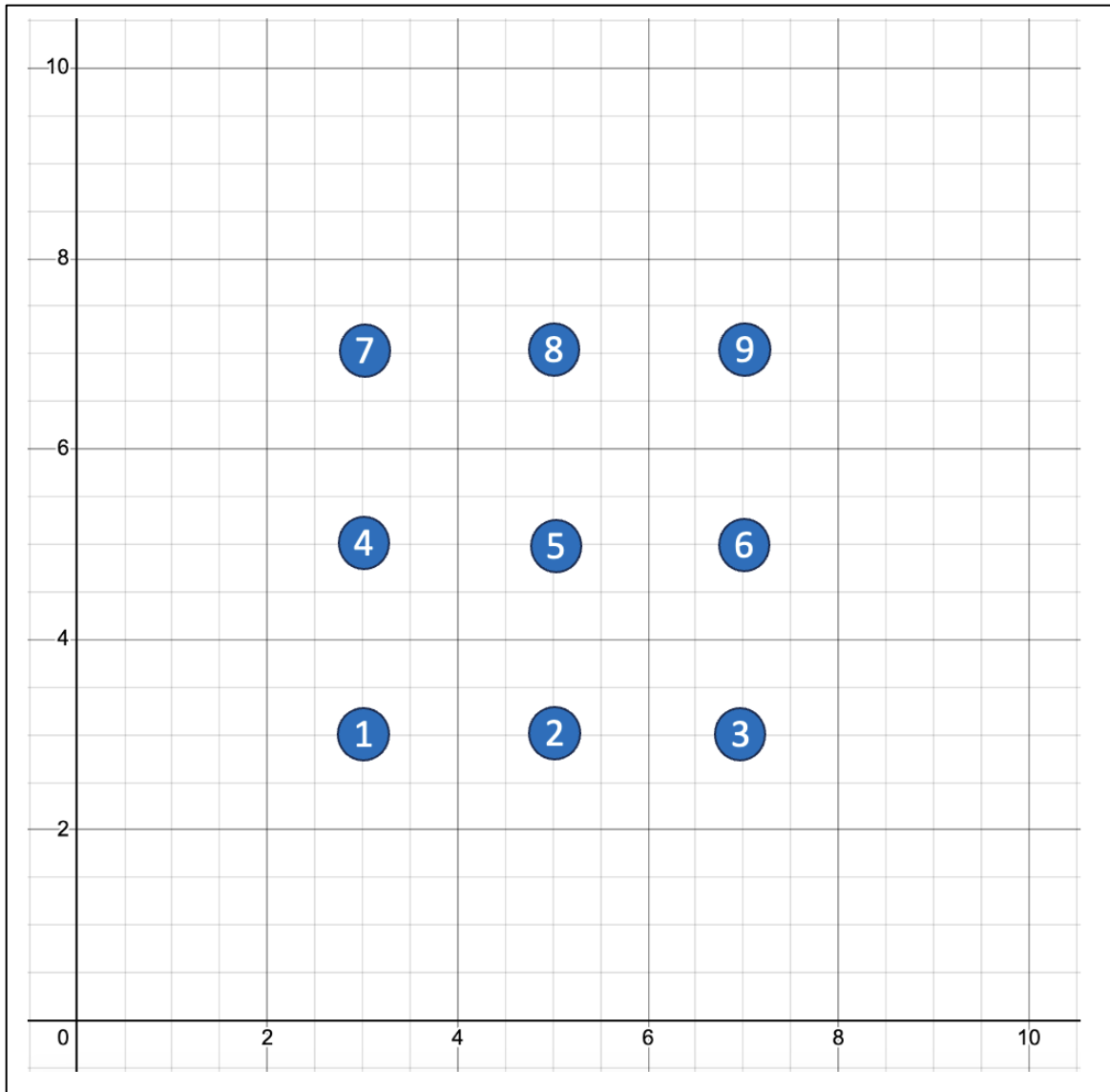
2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Set up a table with sample vectors.

- a. Our first task will be to create a table to store vectors that map to X and Y coordinates as illustrated in the figure below.



- b. Create a table to store the vectors.

```
SQL> CREATE TABLE IF NOT EXISTS vtl
      (id    NUMBER NOT NULL,
       v     VECTOR(2, FLOAT32),
       PRIMARY KEY (id)
      );
```

Table created.

```
SQL> desc vt1
```

| Name | Null?    | Type               |
|------|----------|--------------------|
| ID   | NOT NULL | NUMBER             |
| V    |          | VECTOR(2, FLOAT32) |

```
SQL>
```

- c. Insert the nine vectors into the table vt1.

```
SQL> INSERT INTO vt1
      VALUES (1, '[3, 3]'), (2, '[5, 3]'), (3, '[7, 3]'),
             (4, '[3, 5]'), (5, '[5, 5]'), (6, '[7, 5]'),
             (7, '[3, 7]'), (8, '[5, 7]'), (9, '[7, 7]');
```

9 rows created.

```
SQL> COMMIT;
```

Commit complete.

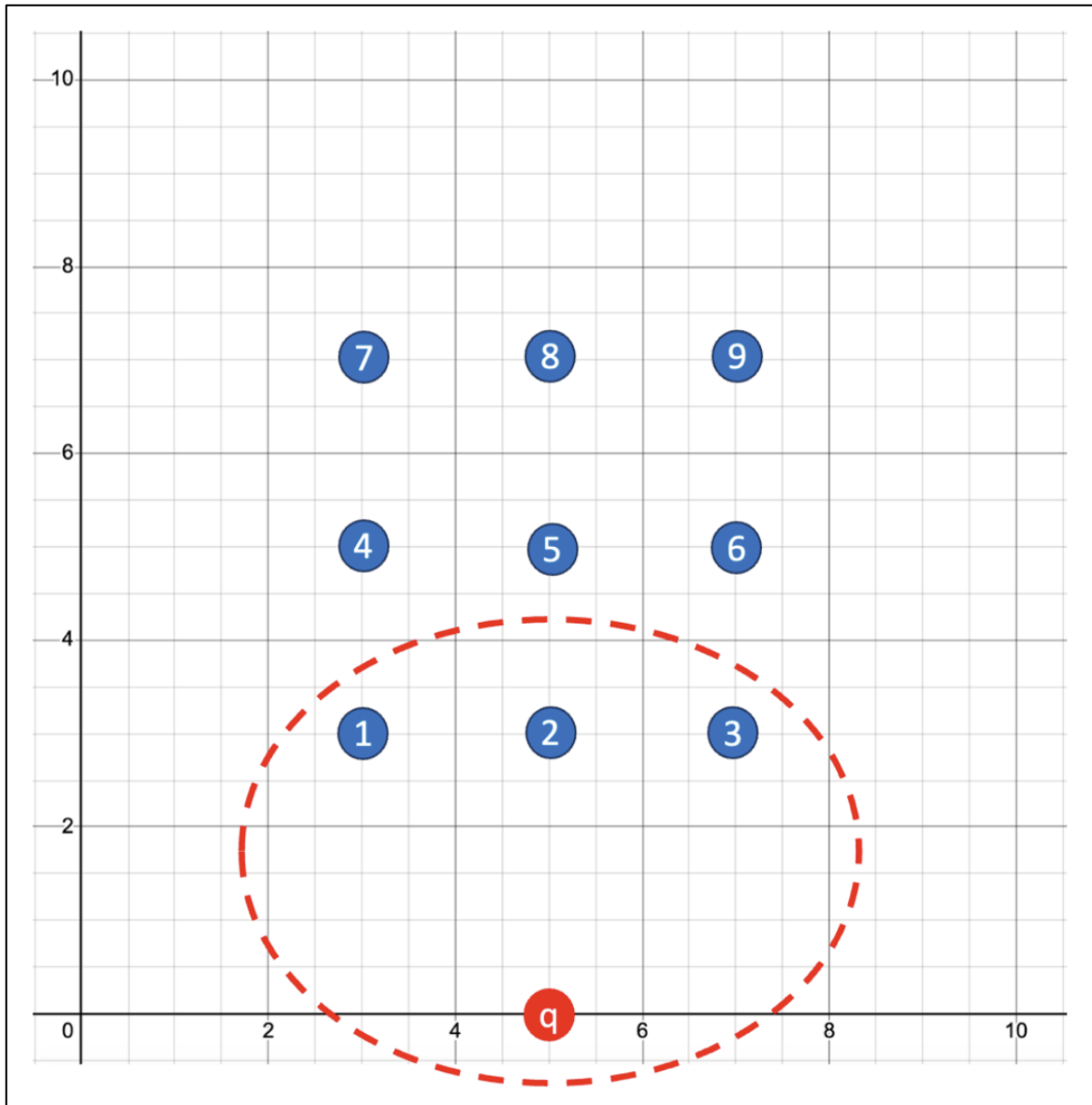
```
SQL> SELECT * FROM vt1
      ORDER BY id;
```

| ID | V                   |
|----|---------------------|
| 1  | [3.0E+000,3.0E+000] |
| 2  | [5.0E+000,3.0E+000] |
| 3  | [7.0E+000,3.0E+000] |
| 4  | [3.0E+000,5.0E+000] |
| 5  | [5.0E+000,5.0E+000] |
| 6  | [7.0E+000,5.0E+000] |
| 7  | [3.0E+000,7.0E+000] |
| 8  | [5.0E+000,7.0E+000] |
| 9  | [7.0E+000,7.0E+000] |

9 rows selected.

```
SQL>
```

4. Look for the closest vectors to  $(5, 0)$ .
- a. In our first example we will look for the three closest vectors to  $(5, 0)$  indicated by the red “q” in the figure below.



For Instructor Use Only.  
This document should not be distributed.

- b. Run the query to find the closest vectors. **Note:** ID 2 should be first.

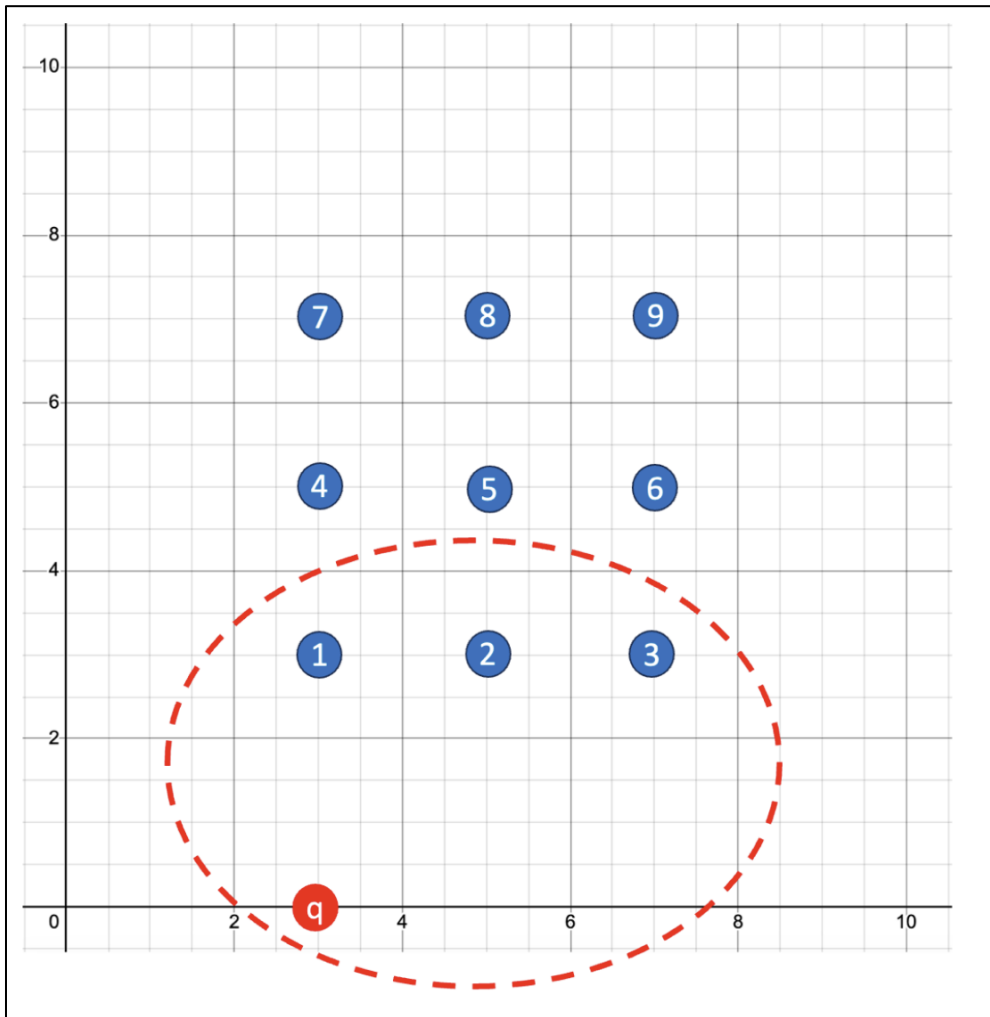
```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(
                vector('[5, 0]'), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

```
      ID
-----
        2
        1
        3
```

3 rows selected.

```
SQL>
```

5. Look for the closest vectors to (3, 0).
  - a. Look for the three closest vectors to (3,0) indicated by the red “q” in the figure below.



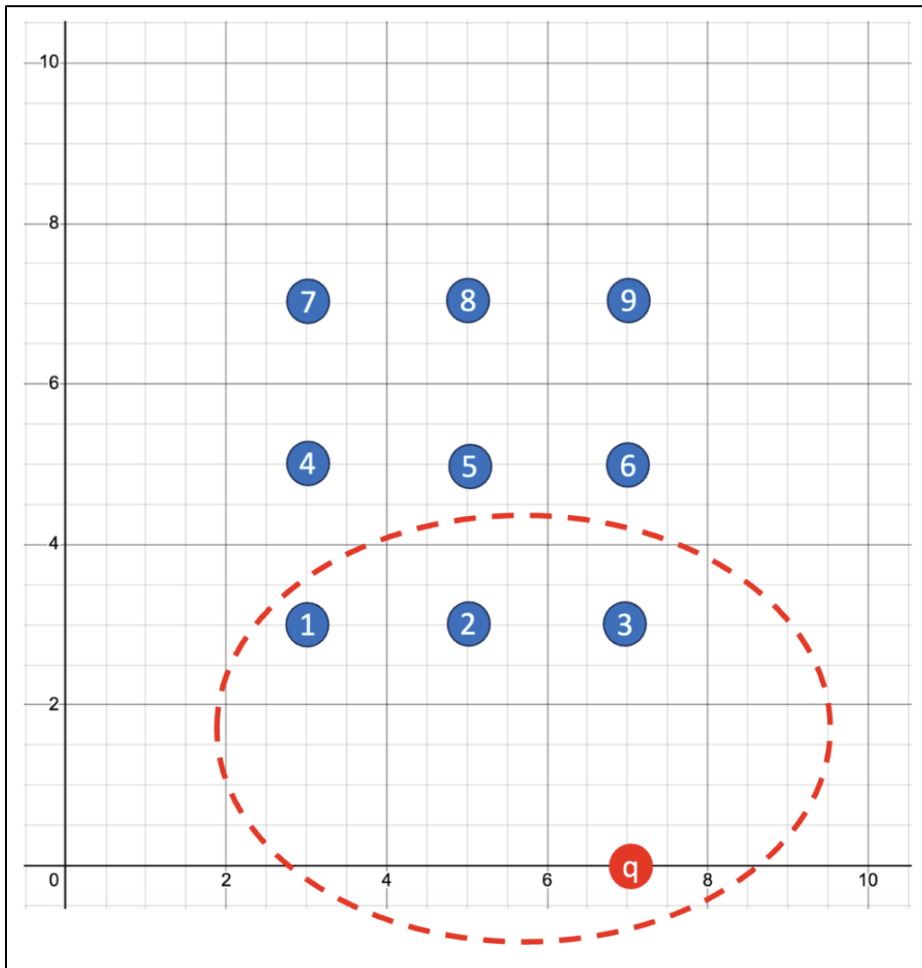
- b. Run the query to find the closest vectors. **Note:** This time ID 1 is the closest and ID 3 is the furthest away, so the answer is expected to be: 1, 2, 3.

```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector(' [3, 0] '), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

| ID |
|----|
| 1  |
| 2  |
| 3  |

```
SQL>
```

6. Look for the closest vectors to (7, 0).
  - a. Look for the three closest vectors to (7,0) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** This time ID 3 is the closest and ID 1 is the furthest away, so the answer is expected to be: 3, 2, 1.

```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector('[7, 0]'), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

```
      ID
-----
        3
        2
        1
```

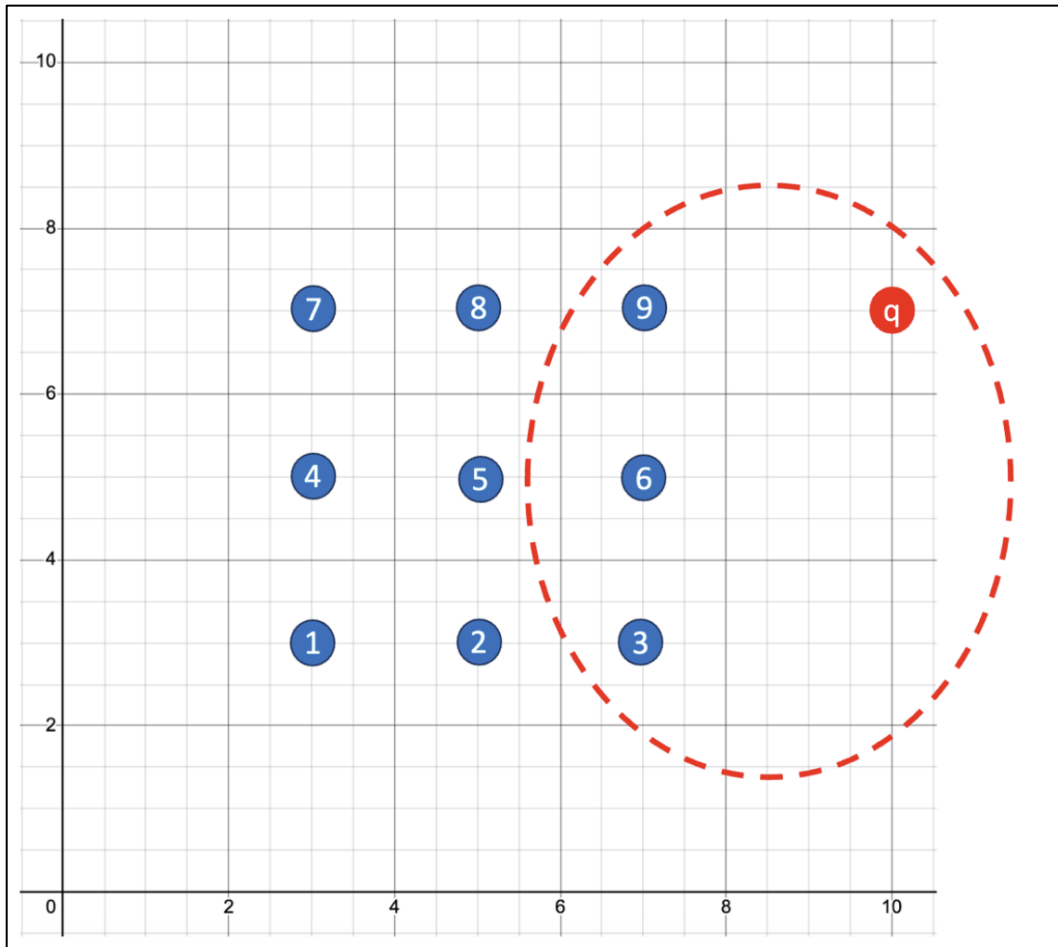
3 rows selected.

```
SQL>
```



7. Look for the closest vectors to (10, 7).

- a. Let's look for the three closest vectors to (10,7) indicated by the red "q" in the figure below.



- b. Run the query to find the closest vectors. **Note:** This time ID 9 is the closest and ID 3 is the furthest away, so the answer is expected to be: 9, 6, 3.

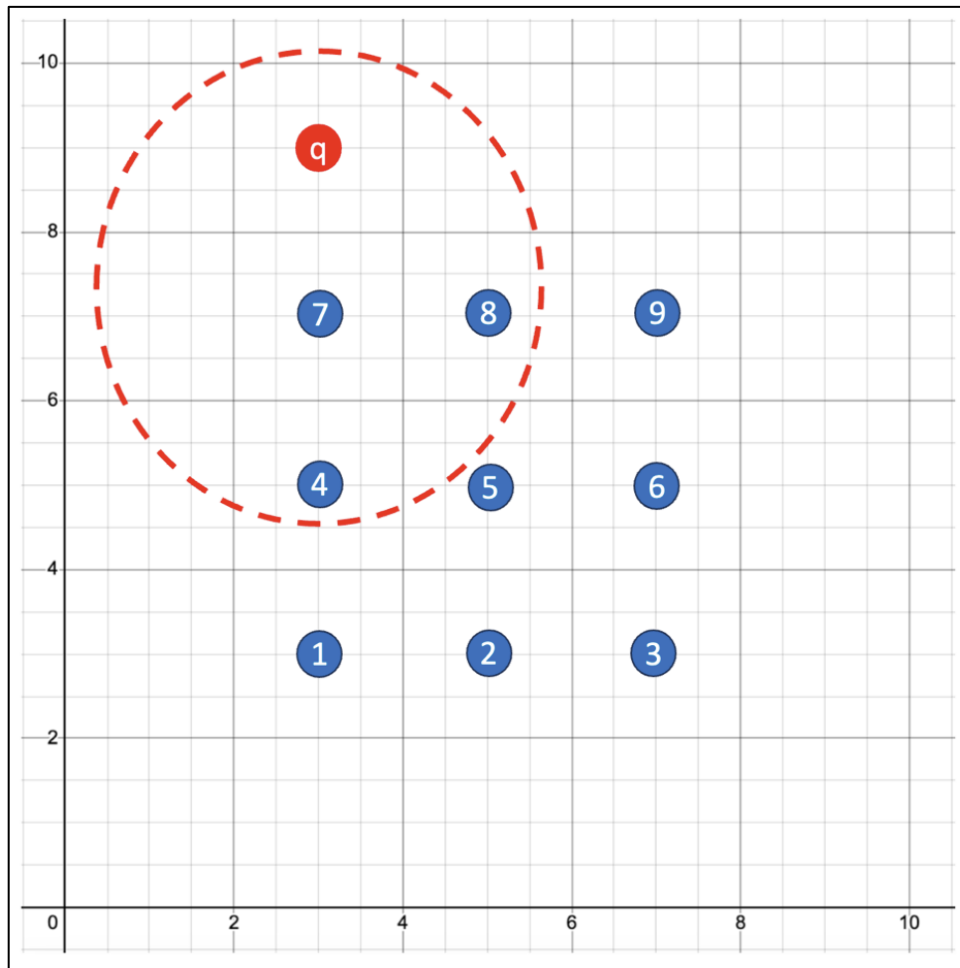
```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector(' [10, 7] '), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

```
      ID
-----
        9
        6
        3
```

3 rows selected.

```
SQL>
```

8. Look for the closest vectors to (3, 9).
  - a. Look for the three closest vectors to (3,9) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** This time ID 7 is the closest and ID 4 is the furthest away, so the answer is expected to be: 7, 8, 4.

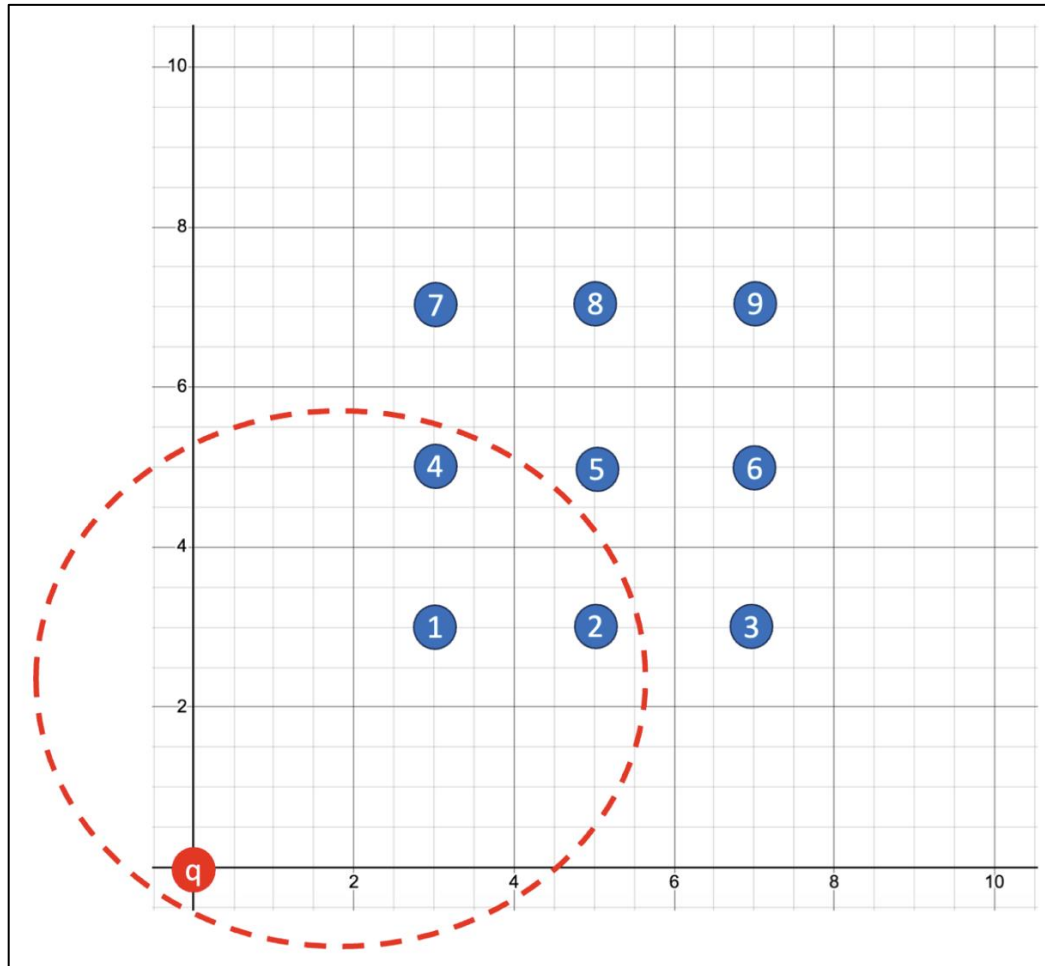
```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector(' [3, 9] '), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

```
      ID
-----
        7
        8
        4
```

3 rows selected.

```
SQL>
```

9. Look for the closest vectors to (0, 0).
  - a. Look for the three closest vectors to (0,0) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** This time ID 1 is the closest and IDs 2 and 4 are the same distance away, so the answer is expected to be either: 1, 4, 2 or 1, 2, 4.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector(' [0, 0] '), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

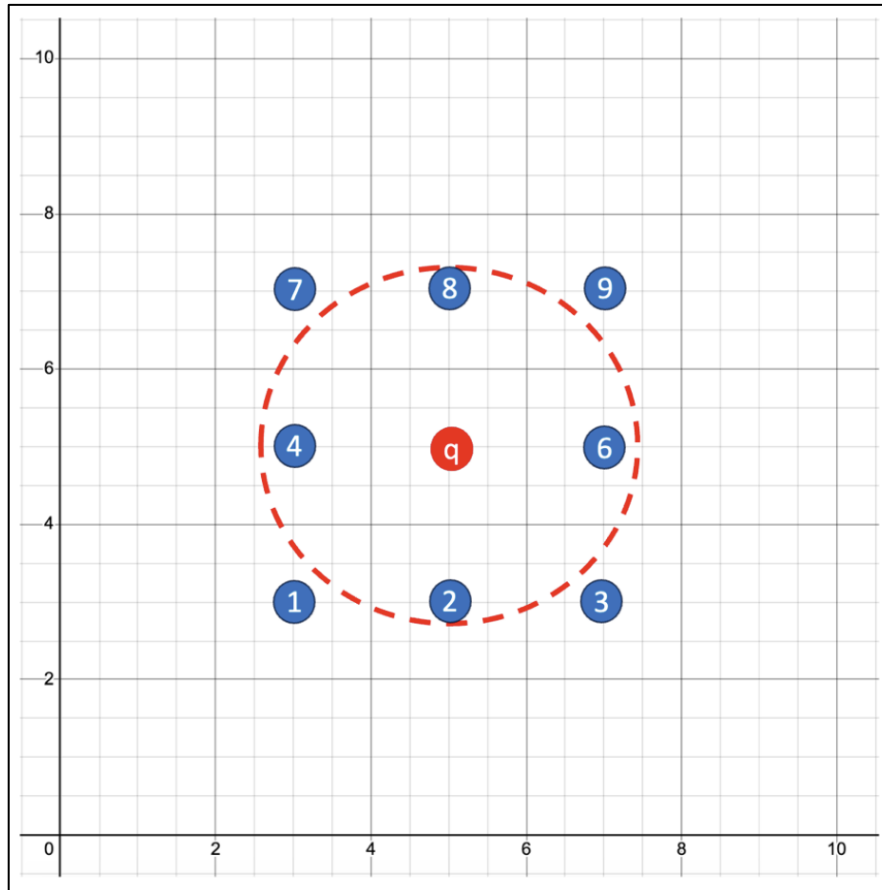
| ID |
|----|
| 1  |
| 2  |
| 4  |

3 rows selected.

SQL>

10. Look for the closest vectors to (5, 5).

a. Look for the five closest vectors to (5,5) indicated by the red “q” in the figure below.



b. Run the query to find the closest vectors. **Note:** This time ID 5 is the closest and IDs 2, 4, 6 and 8 are the same distance away, so the answer is expected to be: first a 5 followed by 2, 4, 6, 8 in any order.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector(' [5, 5] '), v, EUCLIDEAN)
      FETCH FIRST 5 ROWS ONLY;
```

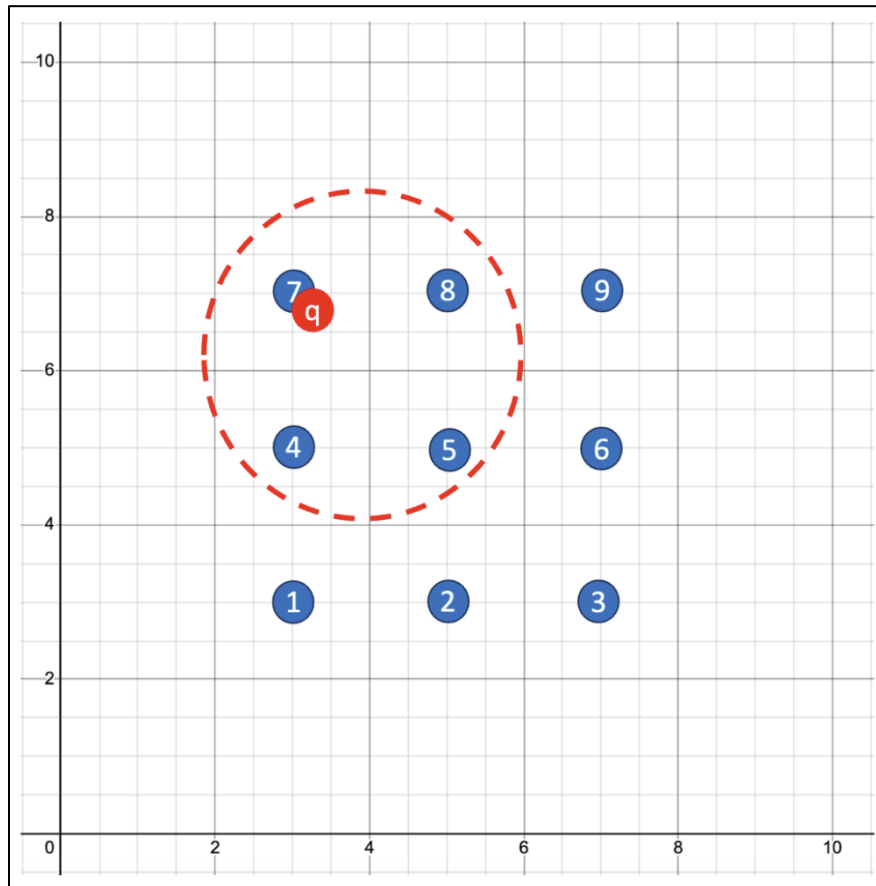
| ID |
|----|
| 5  |
| 2  |
| 8  |
| 6  |
| 4  |

5 rows selected.

SQL>

11. Look for the closest vectors to (3.1, 6.9).

a. Look for the four closest vectors to (3.1,6.9) indicated by the red “q” in the figure below.



b. Run the query to find the closest vectors. **Note:** This time ID 7 is the closest and ID 5 is the furthest distance away, so the answer is expected to be: 7, 4, 8, 5 or 7, 8, 4, 5.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector('[3.1, 6.9]'), v, EUCLIDEAN)
      FETCH FIRST 4 ROWS ONLY;
```

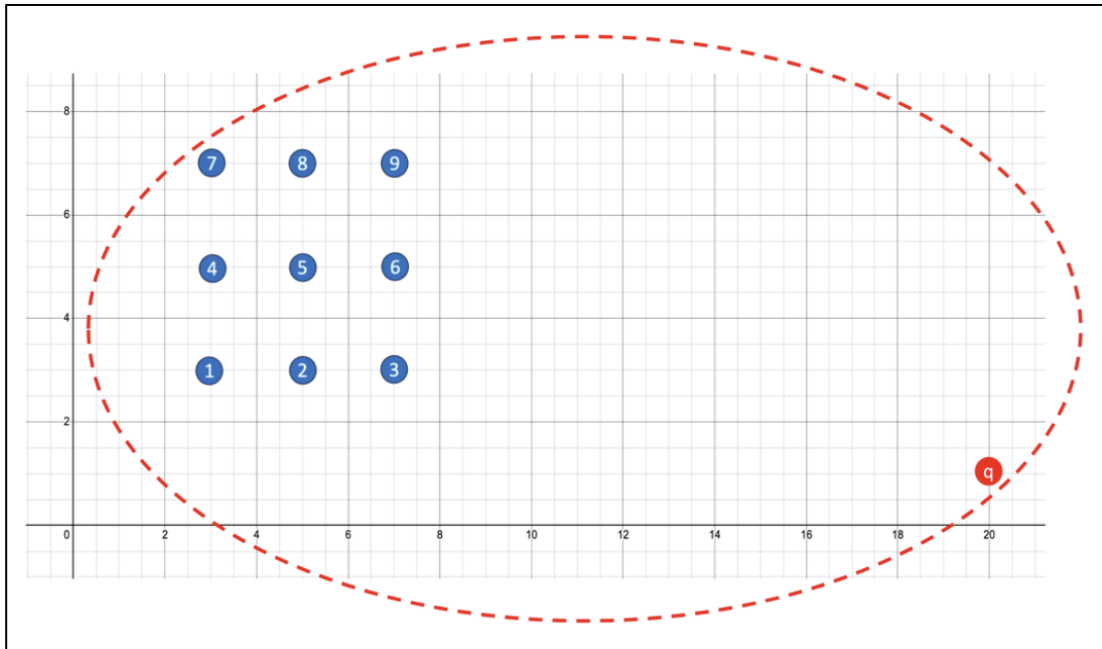
| ID |
|----|
| 7  |
| 4  |
| 8  |
| 5  |

4 rows selected.

SQL>

12. Look for the closest vectors to (20, 1).

- a. Rank all of the vectors in order of proximity to (20, 1) indicated by the red "q" in the figure below.



- b. Run the query to find the closest vectors. **Note:** This time ID 3 is the closest and ID 7 is the furthest distance away, so the answer is expected to start with ID 3 and end with ID 7.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector('[20, 1]'), v, EUCLIDEAN)
      FETCH FIRST 10 ROWS ONLY;

ID
-----
3
6
9
2
5
8
1
4
7

9 rows selected.

SQL>
```

- c. Close the terminal window.

## Practice 9-2: Finding Vectors Based on Vectors Clustered in Groups

---

### Overview

In this practice, you will look for closest vectors based on vectors clustered in groups.

So far, you learned that the `vector_distance` result set has been dependent on two things:

1. Where the query vector was, relative to the cluster, centered around a specific vector or coordinate i.e. (5,5)
2. How many rows were allowed in the result set

However, vectorized data often consists of a set of clusters of vectors and the data tends to not be uniformly distributed. There may also be zero or more clusters of data. Hence, clusters of vectors tend to represent groups of similar data. For example, addresses, types of cars, people names, reports, books, etc.

### Assumptions

You have completed Practices I-1, I-2, I-3, and 9-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

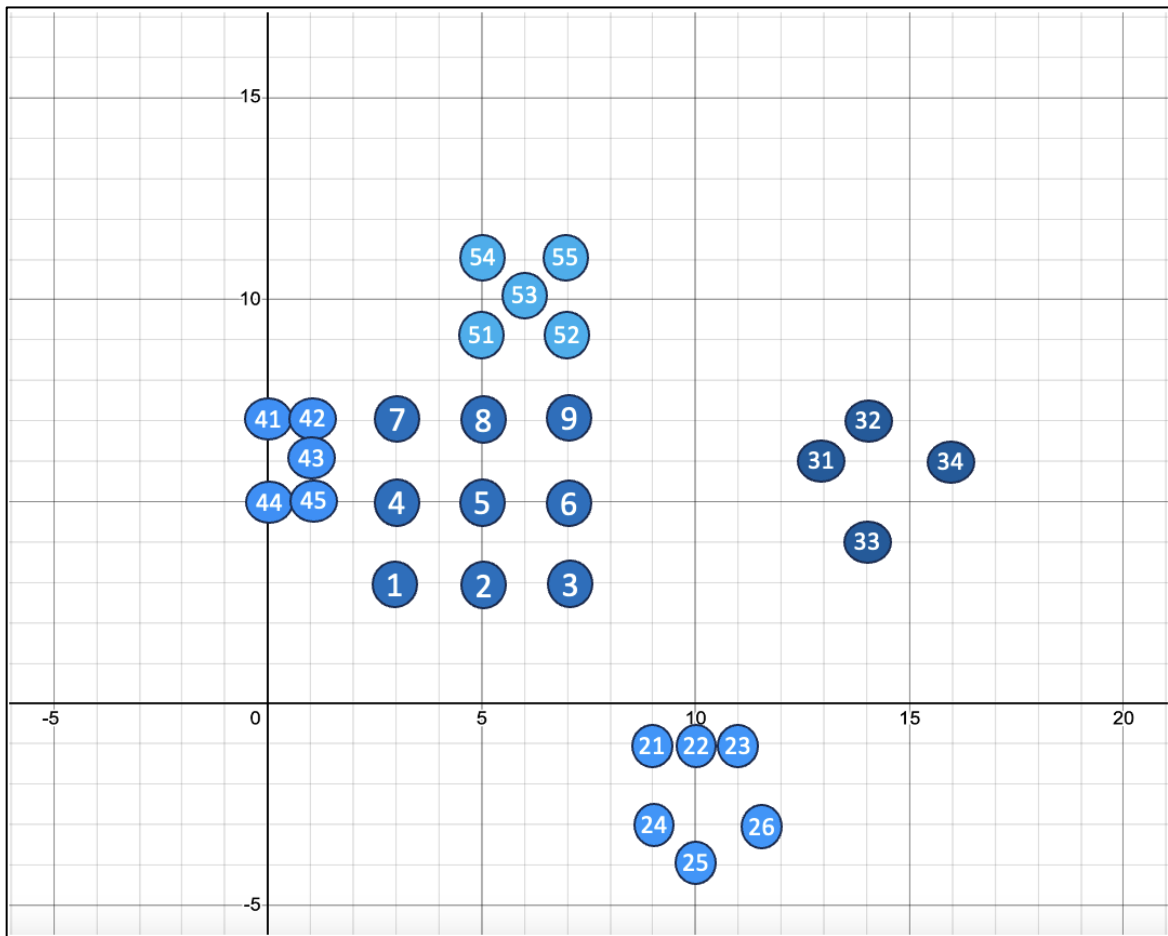
Last Successful login time: Tue May 14 2024 20:46:30 +00:00

Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Add the four clusters of vectors to the table `vt1`.
  - a. You will test with five clusters of vectors like the figure below.



- b. Add four more clusters of vectors to the table `vt1`.

```
SQL> INSERT INTO vt1 VALUES (21, '[9, -1]'),
                             (22, '[10, -1]'),
                             (23, '[11, -1]'),
                             (24, '[9, -3]'),
                             (25, '[10, -4]'),
                             (26, '[12, -3]') ;
```

6 rows created.

```
SQL> INSERT INTO vt1 VALUES (31, '[13, 6]'),
                             (32, '[14, 7]'),
                             (33, '[14, 4]'),
                             (34, '[16, 6]') ;
```

4 rows created.



```
SQL> INSERT INTO vt1 VALUES (41, '[0, 7]'),
                             (42, '[1, 7]'),
                             (43, '[1, 6]'),
                             (44, '[0, 5]'),
                             (45, '[1, 5]') ;
```

5 rows created.

```
SQL> INSERT INTO vt1 VALUES (51, '[5, 9]'),
                             (52, '[7, 9]'),
                             (53, '[6, 10]'),
                             (54, '[5, 11]'),
                             (55, '[7, 11]') ;
```

5 rows created.

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM vt1;
```

| ID    | V                    |
|-------|----------------------|
| ----- |                      |
| 1     | [3.0E+000,3.0E+000]  |
| 2     | [5.0E+000,3.0E+000]  |
| 3     | [7.0E+000,3.0E+000]  |
| 4     | [3.0E+000,5.0E+000]  |
| 5     | [5.0E+000,5.0E+000]  |
| 6     | [7.0E+000,5.0E+000]  |
| 7     | [3.0E+000,7.0E+000]  |
| 8     | [5.0E+000,7.0E+000]  |
| 9     | [7.0E+000,7.0E+000]  |
| 21    | [9.0E+000,-1.0E+000] |
| 22    | [1.0E+001,-1.0E+000] |
| 23    | [1.1E+001,-1.0E+000] |
| 24    | [9.0E+000,-3.0E+000] |
| 25    | [1.0E+001,-4.0E+000] |
| 26    | [1.2E+001,-3.0E+000] |
| 31    | [1.3E+001,6.0E+000]  |
| 32    | [1.4E+001,7.0E+000]  |

```

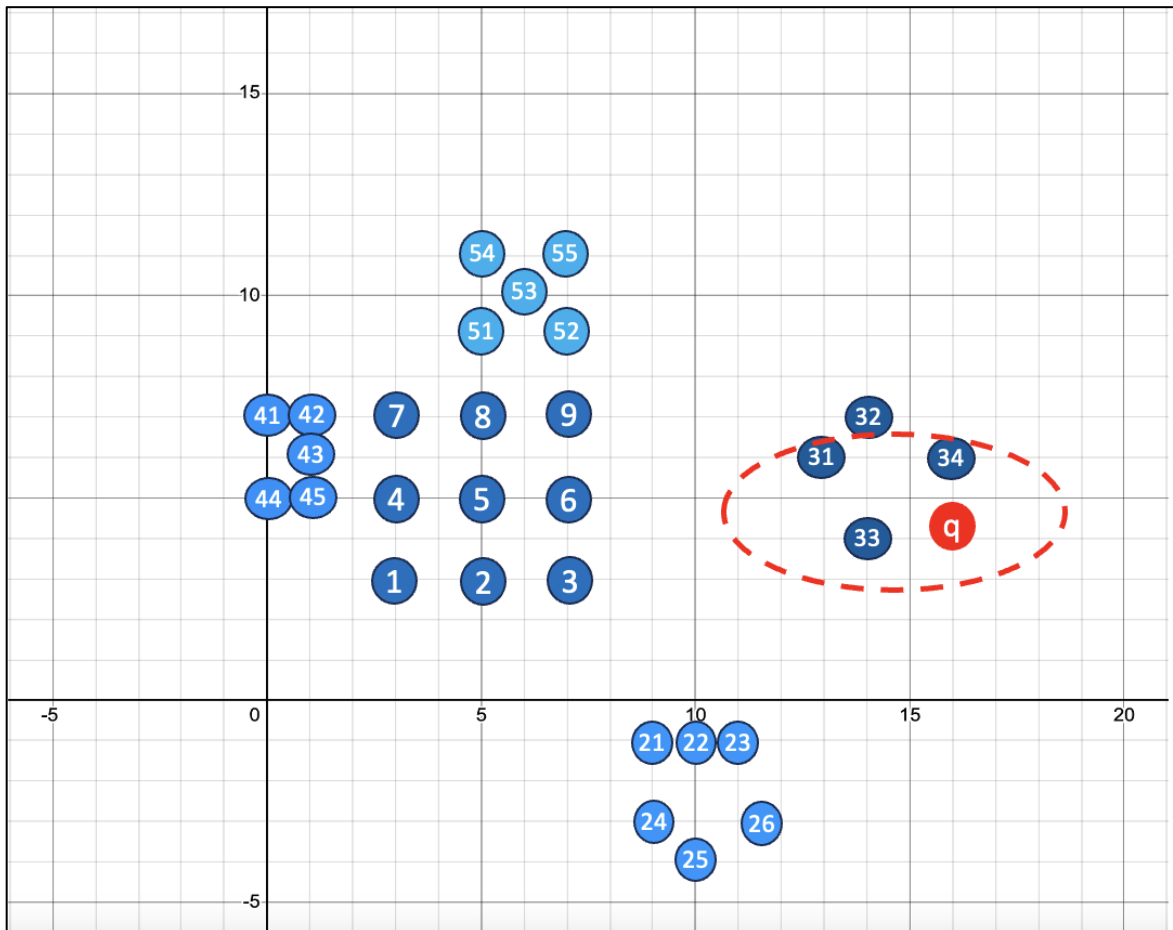
33 [1.4E+001,4.0E+000]
34 [1.6E+001,6.0E+000]
41 [0,7.0E+000]
42 [1.0E+000,7.0E+000]
43 [1.0E+000,6.0E+000]
44 [0,5.0E+000]
45 [1.0E+000,5.0E+000]
51 [5.0E+000,9.0E+000]
52 [7.0E+000,9.0E+000]
53 [6.0E+000,1.0E+001]
54 [5.0E+000,1.1E+001]
55 [7.0E+000,1.1E+001]

```

29 rows selected.

SQL>

4. Look for the closest vectors to (16, 4).
  - a. Look for the three closest vectors to (16,4) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs in the 30s.

```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector('[16, 4]'), v, EUCLIDEAN)
      FETCH FIRST 3 ROWS ONLY;
```

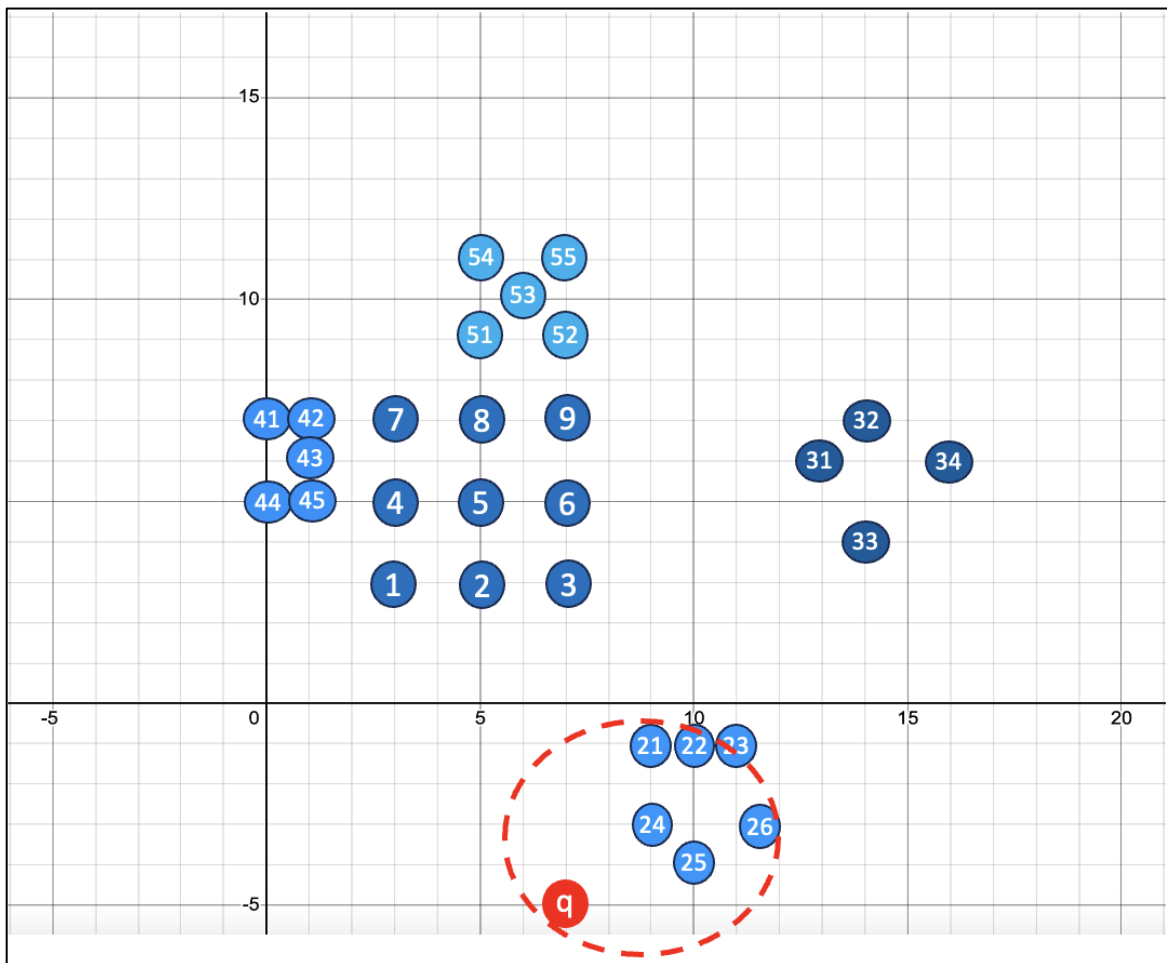
```
      ID
-----
      33
      34
      31
```

3 rows selected.

SQL>

5. Look for the closest vectors to (7, -5).

- a. Look for the five closest vectors to (7,-5) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs in the 20s.

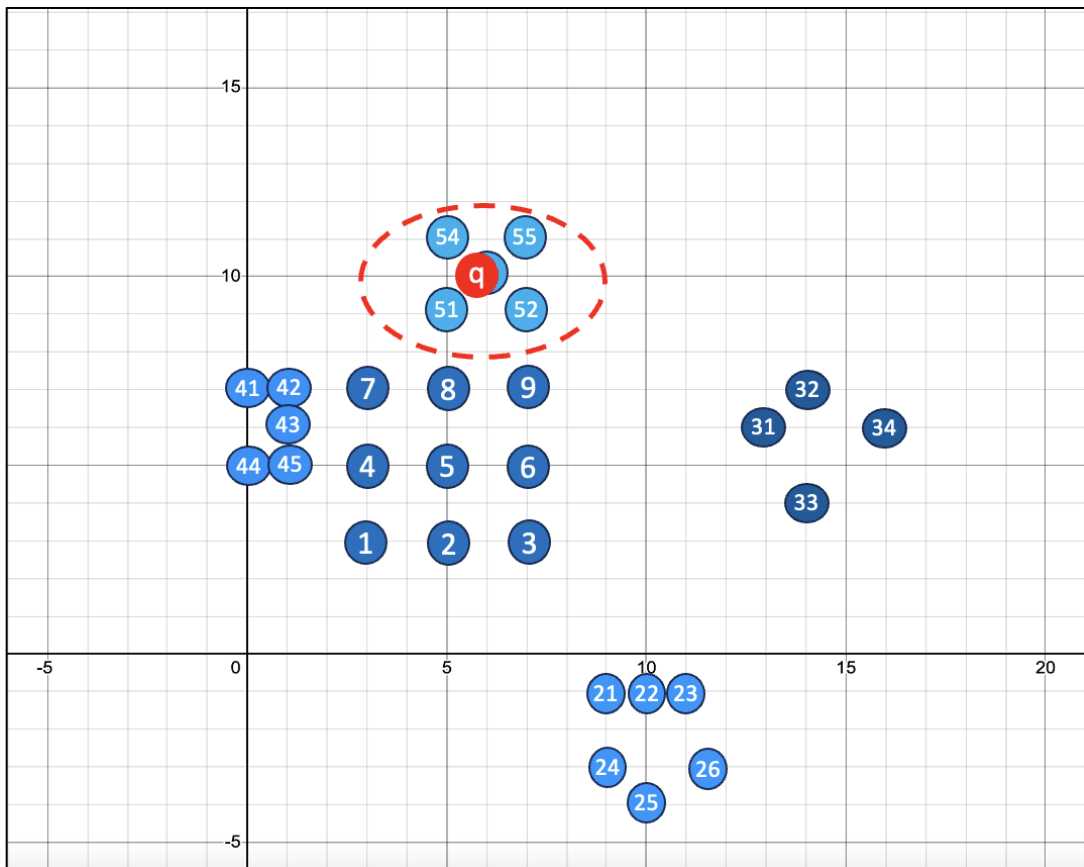
```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector('[7, -5]'), v, EUCLIDEAN)
      FETCH FIRST 5 ROWS ONLY;
```

```
      ID
-----
      24
      25
      21
      22
      26
```

5 rows selected.

```
SQL>
```

6. Look for the closest vectors to (6, 10).
- a. Look for the five closest vectors to (6,10) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs in the 50s, starting with ID 53.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector('[6, 10]'), v, EUCLIDEAN)
      FETCH FIRST 5 ROWS ONLY;
```

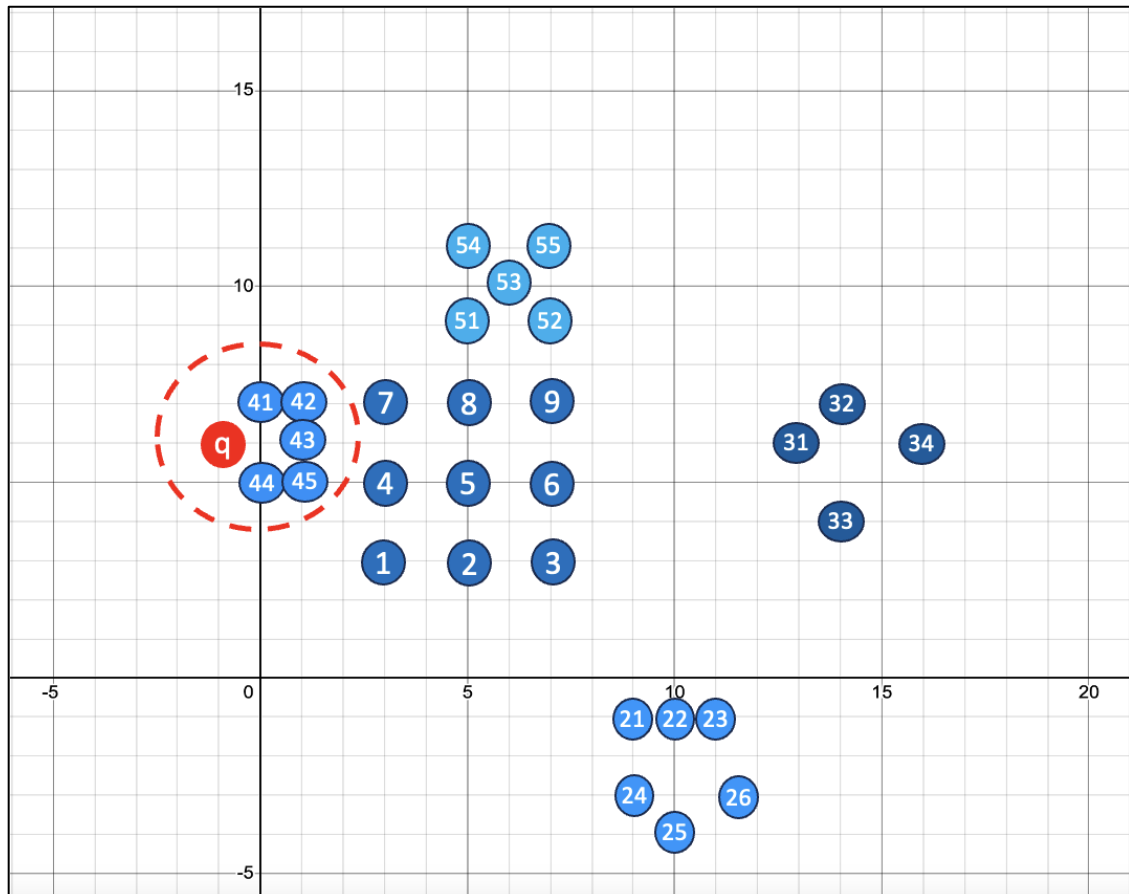
| ID |
|----|
| 53 |
| 51 |
| 55 |
| 54 |
| 52 |

5 rows selected.

SQL>

7. Look for the closest vectors to (-1, 6).

- a. Look for the five closest vectors to (-1,6) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs in the 40s.

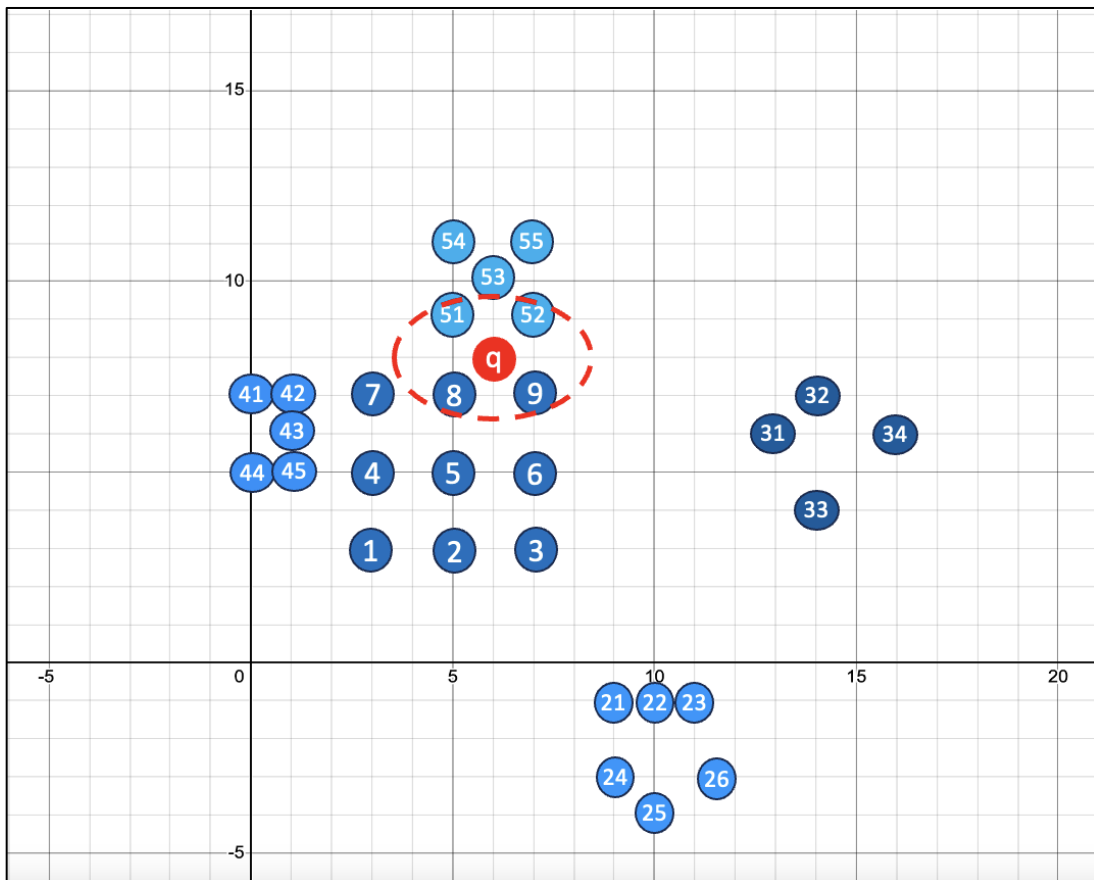
```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector('[-1, 6]'), v, EUCLIDEAN)
      FETCH FIRST 5 ROWS ONLY;
```

| ID |
|----|
| 41 |
| 44 |
| 43 |
| 42 |
| 45 |

5 rows selected.

SQL>

8. Look for the closest vectors to (6, 8).
- a. Look for the four closest vectors to (6,8) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs: 8, 9, 51, and 52.

```
SQL> SELECT id
      FROM vt1
      ORDER BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
      FETCH FIRST 4 ROWS ONLY;
```

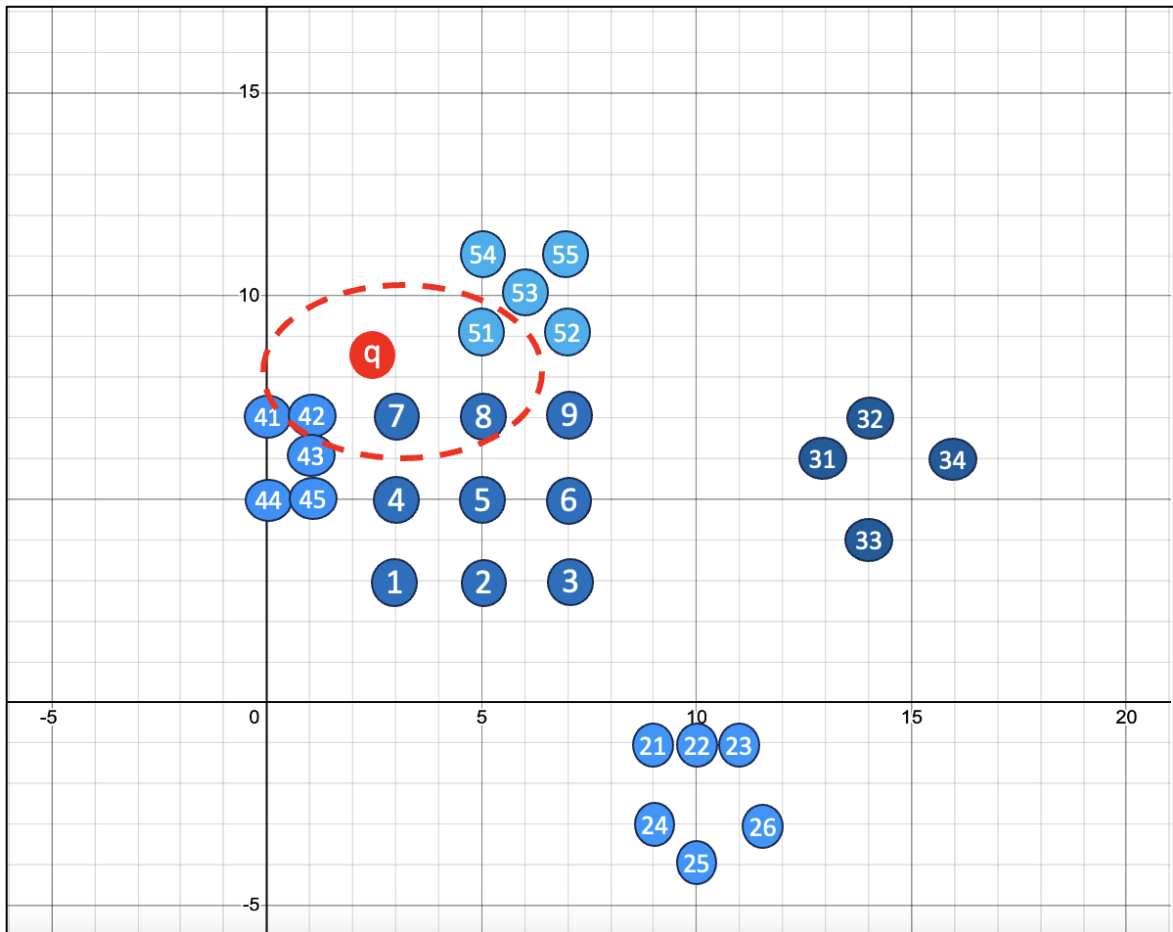
```
      ID
-----
      8
     52
     51
      9

4 rows selected.

SQL>
```

9. Look for the closest vectors to (2.5, 8.5).

- a. Look for the four closest vectors to (2.5,8.5) indicated by the red “q” in the figure below.



- b. Run the query to find the closest vectors. **Note:** We expect to see IDs in the 10s, 40s, and 50s.

```
SQL> SELECT id
      FROM   vt1
      ORDER BY vector_distance(vector('[2.5, 8.5]'), v, EUCLIDEAN)
      FETCH FIRST 4 ROWS ONLY;

          ID
-----
          7
         42
         51
          8

4 rows selected.

SQL>
```

10. Close the terminal window.



**Practices for Lesson 10:  
Narrowing Search Results**

For Instructor Use Only.  
This document should not be distributed.

## Practice 10-1: Setting Up a Table with Sample Vectors

---

### Overview

In this practice, you set up a table with sample vectors to prepare for Attribute Filtering.

### Assumption

You have completed Practices 1-1, 1-2, 1-3, 9-1, and 9-2 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

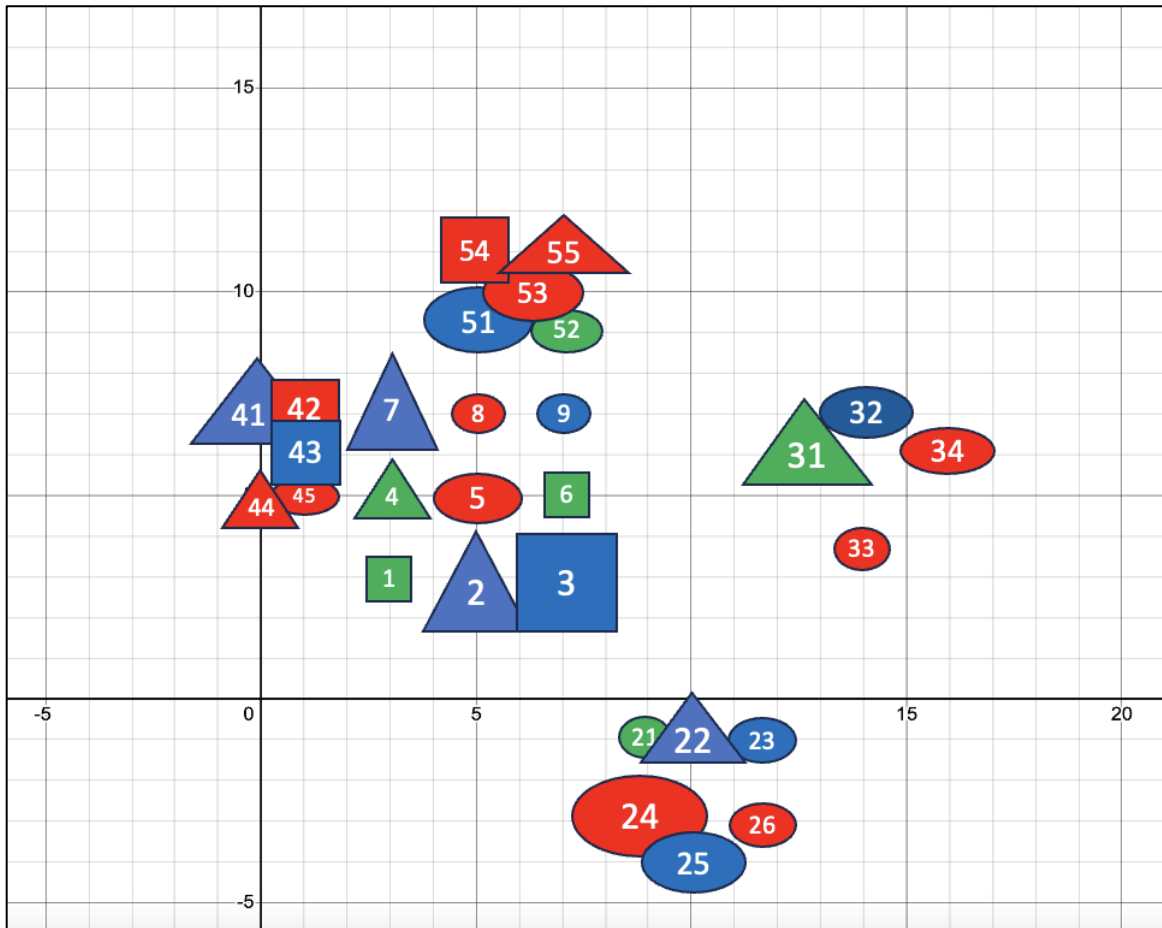
2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WELCOME123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Create a table with sample vectors.

- a. You will create a table called `vt2` with vectors to map different shapes, colors, and sizes as illustrated in the figure below.



- b. Create a table to store the vectors.

```
SQL> DROP TABLE IF EXISTS vt2;
```

Table dropped.

```
SQL> CREATE TABLE vt2 AS SELECT * FROM vt1;
```

Table created.

```
SQL> ALTER TABLE vt2 ADD (vsize varchar2(16),  
                             shape varchar2(16),  
                             color varchar2(16)  
                             );
```

Table altered.

```
SQL> DESC vt2
```

| Name  | Null?    | Type               |
|-------|----------|--------------------|
| ID    | NOT NULL | NUMBER             |
| V     |          | VECTOR(2, FLOAT32) |
| VSIZE |          | VARCHAR2(16)       |
| SHAPE |          | VARCHAR2(16)       |
| COLOR |          | VARCHAR2(16)       |

```
SQL>
```

4. Set the sizes, shapes, and colors in the table that we will be using to query with Attribute Filtering.
  - a. Set up the vector sizes.

```
SQL> UPDATE vt2
```

```
SET      vsize = 'Small'
```

```
WHERE id IN (1, 4, 6, 8, 9, 21, 23, 26, 33, 44, 45, 52);
```

```
12 rows updated.
```

```
SQL> UPDATE vt2
```

```
SET      vsize = 'Medium'
```

```
WHERE id IN (5, 22, 25, 32, 34, 42, 43, 53, 54, 55);
```

```
10 rows updated.
```

```
SQL> UPDATE vt2
```

```
SET      vsize = 'Large'
```

```
WHERE id IN (2, 3, 7, 24, 31, 41, 51);
```

```
7 rows updated.
```

```
SQL>
```

- b. Set up the vector shapes.

```
SQL> UPDATE vt2
```

```
SET      shape = 'Square'
```

```
WHERE id IN (1, 3, 6, 42, 43, 54);
```

```
6 rows updated.
```

```

SQL> UPDATE vt2
SET     shape = 'Triangle'
WHERE   id IN (2, 4, 7, 22, 31, 41, 44, 55);

8 rows updated.

SQL> UPDATE vt2
SET     shape = 'Oval'
WHERE   id IN (5, 8, 9, 21, 23, 24, 25, 26, 32, 33, 34, 45, 51,
52, 53);

15 rows updated.

SQL>

```

- c. Set up the vector colors.

```

SQL> UPDATE vt2
SET     color = 'Red'
WHERE   id IN (5, 8, 24, 26, 33, 34, 42, 44, 45, 53, 54, 55);

12 rows updated.

SQL> UPDATE vt2
SET     color = 'Green'
WHERE   id IN (1, 4, 6, 21, 31, 52);

6 rows updated.

SQL> UPDATE vt2
SET     color = 'Blue'
WHERE   id IN (2, 3, 7, 9, 22, 23, 25, 32, 41, 43, 51);

11 rows updated.

SQL> COMMIT;

Commit complete.

SQL>

```

- d. Confirm the updates have been correctly applied to table VT2.

```
SQL> SELECT id, vsize, shape, color, v
FROM   vt2
ORDER  BY id;
```

```
SQL> SELECT id, vsize, shape, color, v
2 FROM   vt2
3 ORDER  BY id;
```

| ID | VSIZE  | SHAPE    | COLOR | V                    |
|----|--------|----------|-------|----------------------|
| 1  | Small  | Square   | Green | [3.0E+000,3.0E+000]  |
| 2  | Large  | Triangle | Blue  | [5.0E+000,3.0E+000]  |
| 3  | Large  | Square   | Blue  | [7.0E+000,3.0E+000]  |
| 4  | Small  | Triangle | Green | [3.0E+000,5.0E+000]  |
| 5  | Medium | Oval     | Red   | [5.0E+000,5.0E+000]  |
| 6  | Small  | Square   | Green | [7.0E+000,5.0E+000]  |
| 7  | Large  | Triangle | Blue  | [3.0E+000,7.0E+000]  |
| 8  | Small  | Oval     | Red   | [5.0E+000,7.0E+000]  |
| 9  | Small  | Oval     | Blue  | [7.0E+000,7.0E+000]  |
| 21 | Small  | Oval     | Green | [9.0E+000,-1.0E+000] |
| 22 | Medium | Triangle | Blue  | [1.0E+001,-1.0E+000] |
| 23 | Small  | Oval     | Blue  | [1.1E+001,-1.0E+000] |
| 24 | Large  | Oval     | Red   | [9.0E+000,-3.0E+000] |
| 25 | Medium | Oval     | Blue  | [1.0E+001,-4.0E+000] |
| 26 | Small  | Oval     | Red   | [1.2E+001,-3.0E+000] |
| 31 | Large  | Triangle | Green | [1.3E+001,6.0E+000]  |
| 32 | Medium | Oval     | Blue  | [1.4E+001,7.0E+000]  |
| 33 | Small  | Oval     | Red   | [1.4E+001,4.0E+000]  |
| 34 | Medium | Oval     | Red   | [1.6E+001,6.0E+000]  |
| 41 | Large  | Triangle | Blue  | [0,7.0E+000]         |
| 42 | Medium | Square   | Red   | [1.0E+000,7.0E+000]  |
| 43 | Medium | Square   | Blue  | [1.0E+000,6.0E+000]  |
| 44 | Small  | Triangle | Red   | [0,5.0E+000]         |
| 45 | Small  | Oval     | Red   | [1.0E+000,5.0E+000]  |
| 51 | Large  | Oval     | Blue  | [5.0E+000,9.0E+000]  |
| 52 | Small  | Oval     | Green | [7.0E+000,9.0E+000]  |
| 53 | Medium | Oval     | Red   | [6.0E+000,1.0E+001]  |
| 54 | Medium | Square   | Red   | [5.0E+000,1.1E+001]  |
| 55 | Medium | Triangle | Red   | [7.0E+000,1.1E+001]  |

29 rows selected.

- e. Display a summarized breakdown of the different shapes, sizes, and colors for the table VT2.

```
SQL> SELECT vsize, count(vsize)
FROM    vt2
GROUP   BY vsize;
```

| VSIZE  | COUNT (VSIZE) |
|--------|---------------|
| Small  | 12            |
| Large  | 7             |
| Medium | 10            |

```
SQL> SELECT color, COUNT(color)
FROM    vt2
GROUP   BY color;
```

| COLOR | COUNT (COLOR) |
|-------|---------------|
| Green | 6             |
| Blue  | 11            |
| Red   | 12            |

```
SQL> SELECT shape, COUNT(shape)
FROM    vt2
GROUP   BY shape;
```

| SHAPE    | COUNT (SHAPE) |
|----------|---------------|
| Square   | 6             |
| Triangle | 8             |
| Oval     | 15            |

```
SQL>
```

5. Close the terminal window.

## Practice 10-2: Attribute Filtering

---

### Overview

In this practice, you see how `vector_distance()` can be used in combination with other SQL `WHERE` predicates.

### Assumption

You have completed Practice 10-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to `ORCLPDB1` as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

Last Successful login time: Tue May 14 2024 20:46:30 +00:00

Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

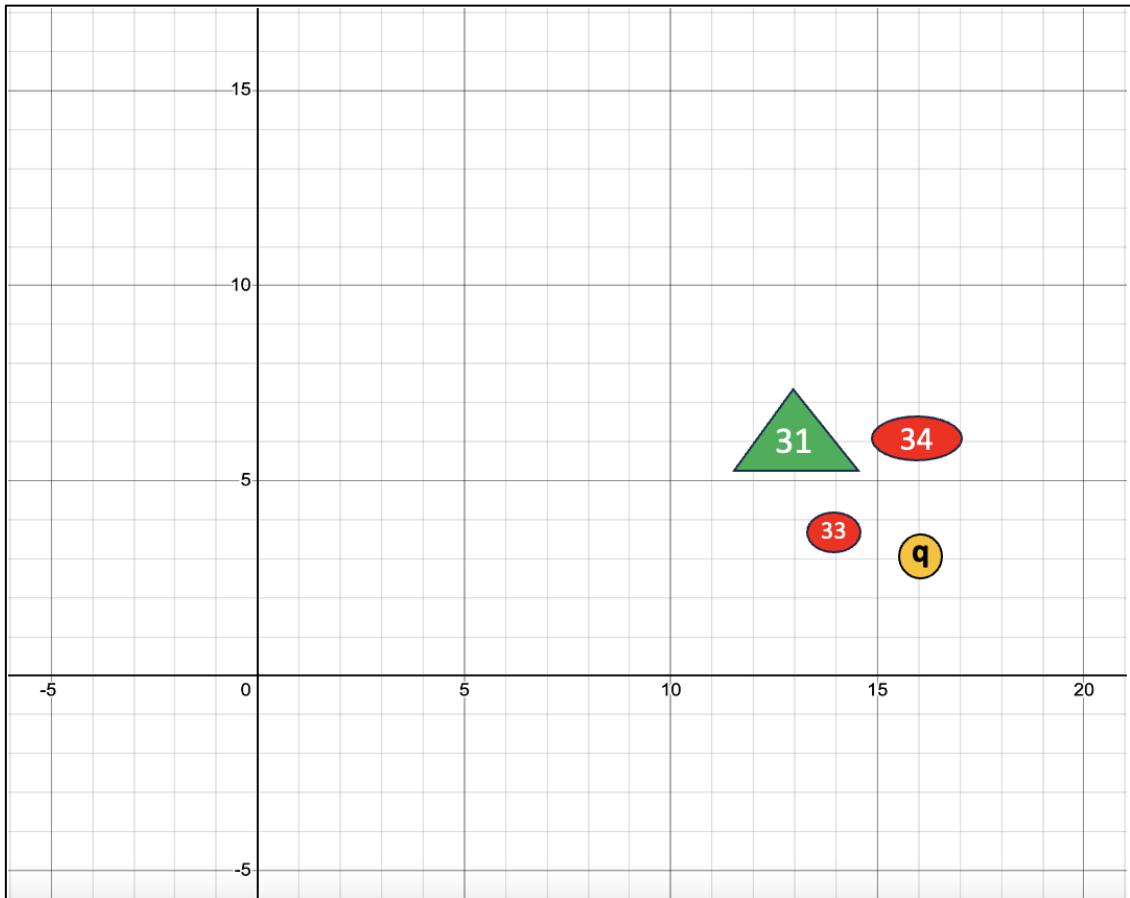
SQL>
```

For Instructor Use Only.  
This document should not be distributed.



3. Find the three closest vectors to (16,3).

- a. Look for the three closest vectors to (16,3) indicated by the yellow “q” in the figure below.



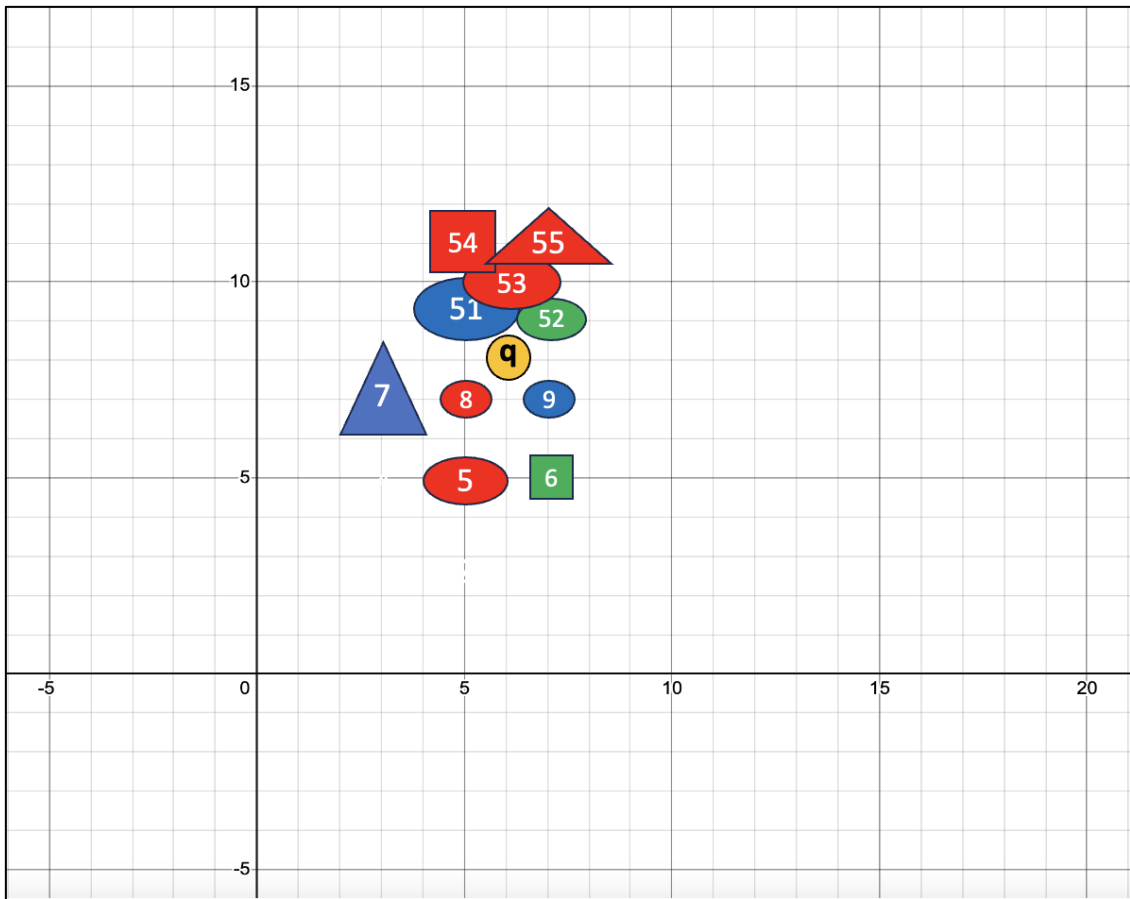
- b. Run the query to retrieve the vectors from the figure above. **Note:** You should notice that the query will artificially return the IDs from a cluster.

```
SQL> SELECT id, vsize, shape, color,
         to_number(vector_distance(vector('[16, 3]'), v,
         EUCLIDEAN)) distance
       FROM vt2
       WHERE id > 30 AND id < 40
       ORDER BY vector_distance(vector('[16, 3]'), v, EUCLIDEAN)
       FETCH FIRST 3 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR | DISTANCE   |
|----|--------|----------|-------|------------|
| 33 | Small  | Oval     | Red   | 2.23606801 |
| 34 | Medium | Oval     | Red   | 3          |
| 31 | Large  | Triangle | Green | 4.2426405  |

SQL>

4. Find the ten closest vectors to (6,8).
  - a. Look for the closest vectors to (6,8) indicated by the yellow “q” in the figure below.



For Instructor Use Only.  
This document should not be distributed.

b. Run the query to retrieve the vectors from the figure above.

```
SQL> SELECT id, vsize, shape, color
FROM vt2
ORDER BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
FETCH FIRST 10 ROWS ONLY;
```

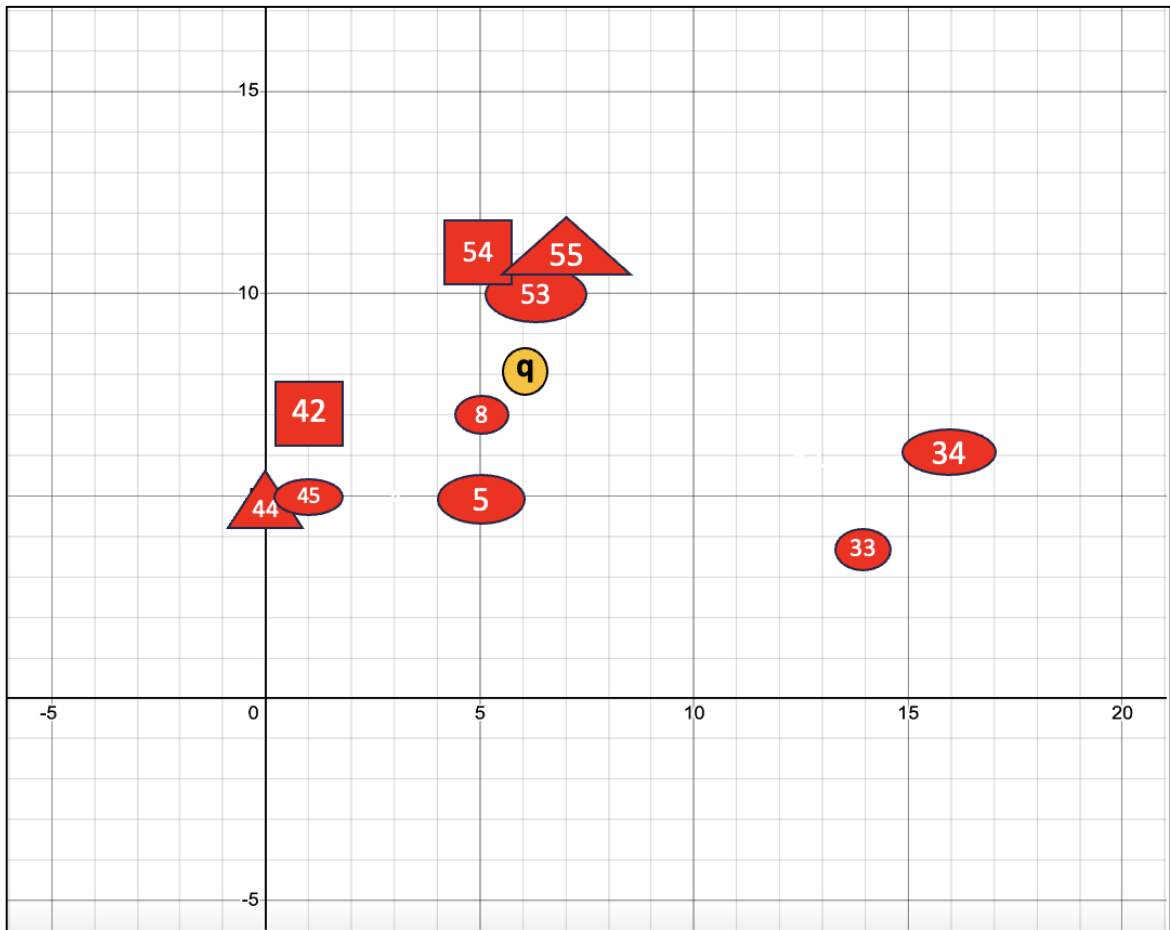
| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 8  | Small  | Oval     | Red   |
| 52 | Small  | Oval     | Green |
| 51 | Large  | Oval     | Blue  |
| 9  | Small  | Oval     | Blue  |
| 53 | Medium | Oval     | Red   |
| 5  | Medium | Oval     | Red   |
| 55 | Medium | Triangle | Red   |
| 54 | Medium | Square   | Red   |
| 7  | Large  | Triangle | Blue  |
| 6  | Small  | Square   | Green |

10 rows selected.

```
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

5. Find the ten closest vectors to (6,8) filtered by color.
- a. Look for the closest vectors to (6,8) indicated by the yellow “q” in the figure below.



For Instructor Use Only.  
This document should not be distributed.

- b. Run the query to retrieve the vectors from the figure above.

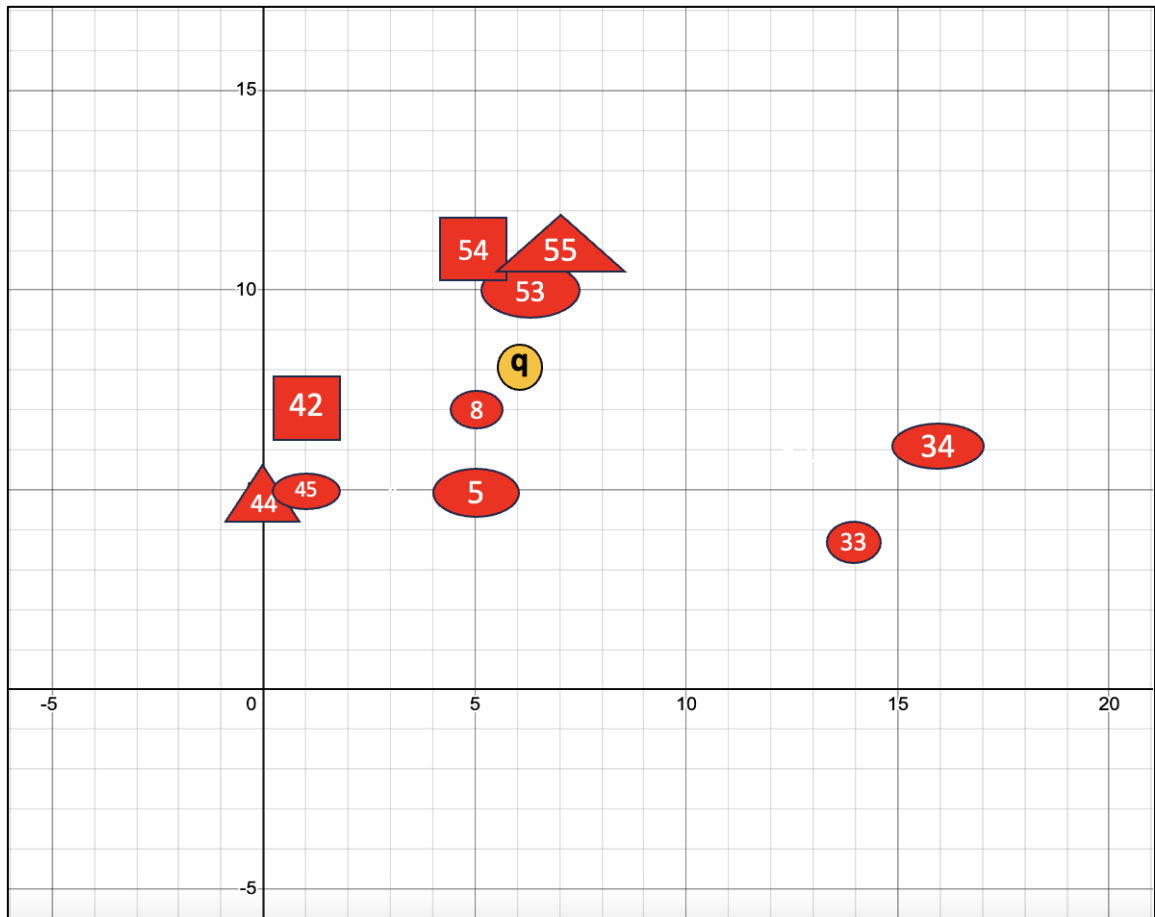
```
SQL> SELECT id, vsize, shape, color
      FROM vt2
      WHERE color = 'Red'
      ORDER BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
      FETCH FIRST 10 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 8  | Small  | Oval     | Red   |
| 53 | Medium | Oval     | Red   |
| 5  | Medium | Oval     | Red   |
| 55 | Medium | Triangle | Red   |
| 54 | Medium | Square   | Red   |
| 42 | Medium | Square   | Red   |
| 45 | Small  | Oval     | Red   |
| 44 | Small  | Triangle | Red   |
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |

10 rows selected.

SQL>

6. Find the ten closest vectors to (6,8) filtered by color and shape.
  - a. Look for the closest vectors to (6,8) indicated by the yellow “q” in the figure below.



For Instructor Use Only.  
This document should not be distributed.

- b. Run the query to retrieve the vectors from the figure above. **Note:** You will also notice that the query only returns eight vectors as there are only eight red ovals in the graph.

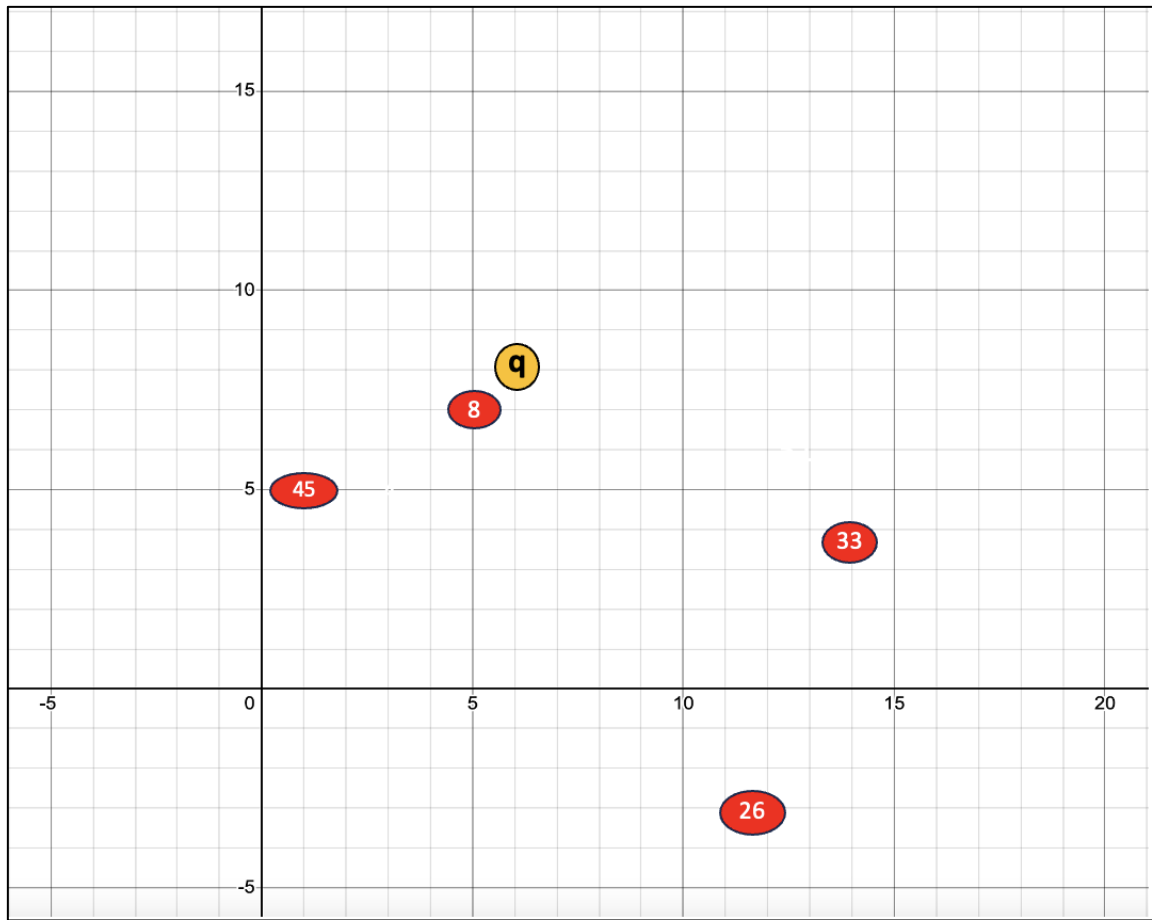
```
SQL> SELECT id, vsize, shape, color
      FROM   vt2
      WHERE  color = 'Red'
      AND    shape = 'Oval'
      ORDER  BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
      FETCH FIRST 10 ROWS ONLY;
```

| ID | VSIZE  | SHAPE | COLOR |
|----|--------|-------|-------|
| 8  | Small  | Oval  | Red   |
| 53 | Medium | Oval  | Red   |
| 5  | Medium | Oval  | Red   |
| 45 | Small  | Oval  | Red   |
| 33 | Small  | Oval  | Red   |
| 34 | Medium | Oval  | Red   |
| 24 | Large  | Oval  | Red   |
| 26 | Small  | Oval  | Red   |

8 rows selected.

SQL>

7. Find the ten closest vectors to (6,8) filtered by color, shape, and size.
- a. Look for the closest vectors to (6,8) indicated by the yellow “q” in the figure below.



For Instructor Use Only.  
This document should not be distributed.



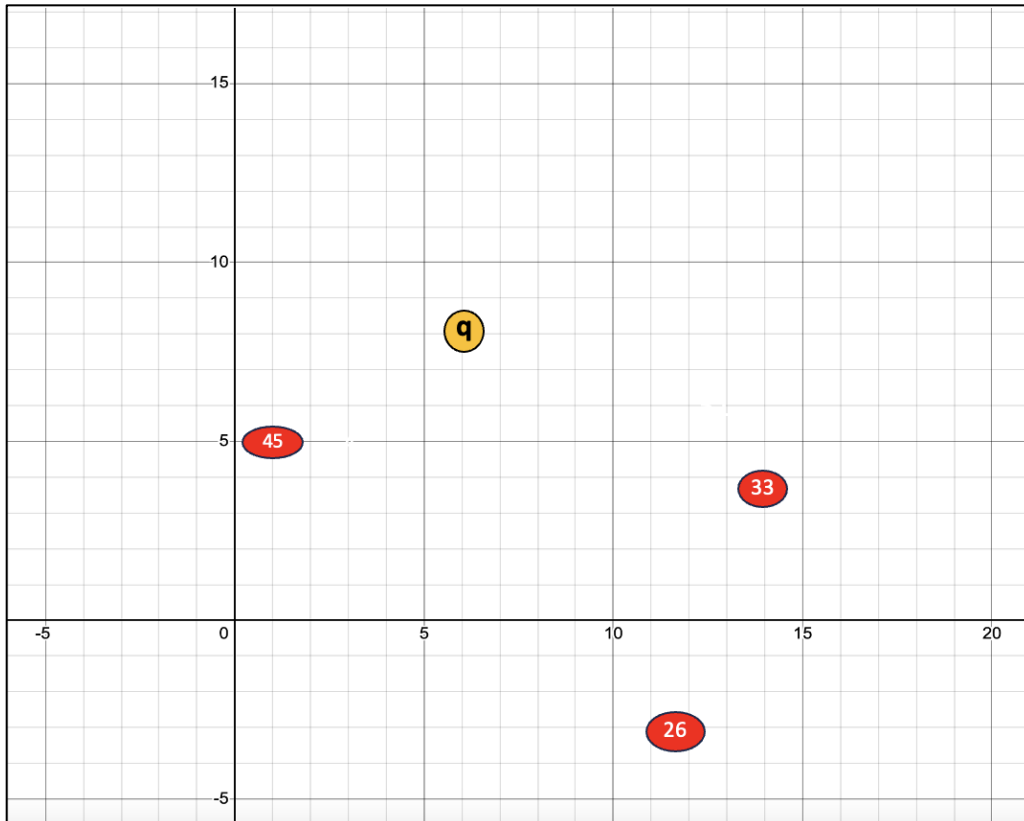
- b. Run the query to retrieve the vectors from the figure above. **Note:** You will also notice that this query only returns four Vectors as there are only four small red ovals in the graph.

```
SQL> SELECT id, vsize, shape, color
      FROM vt2
      WHERE color = 'Red'
      AND shape = 'Oval'
      AND vsize = 'Small'
      ORDER BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
      FETCH FIRST 10 ROWS ONLY;
```

| ID | VSIZE | SHAPE | COLOR |
|----|-------|-------|-------|
| 8  | Small | Oval  | Red   |
| 45 | Small | Oval  | Red   |
| 33 | Small | Oval  | Red   |
| 26 | Small | Oval  | Red   |

SQL>

8. Find the ten closest vectors to (6,8) filtered by color, shape, and size as well as having an ID greater than 10.
- a. Look for the closest vectors to (6,8) indicated by the yellow “q” in the figure below.



- b. Run the query to retrieve the vectors from the figure above. **Note:** You will also notice that this query only returns three vectors as there are only three small red ovals in the graph with ID higher than 10.

```
SQL> SELECT id, vsize, shape, color
      FROM vt2
      WHERE color = 'Red'
      AND shape = 'Oval'
      AND vsize = 'Small'
      AND id > 10
      ORDER BY vector_distance(vector('[6, 8]'), v, EUCLIDEAN)
      FETCH FIRST 10 ROWS ONLY;
```

| ID | VSIZE | SHAPE | COLOR |
|----|-------|-------|-------|
| 45 | Small | Oval  | Red   |
| 33 | Small | Oval  | Red   |
| 26 | Small | Oval  | Red   |

```
SQL>
```

9. Close the terminal window.

**Practices for Lesson 11:  
Testing Other Distance  
Functions**

For Instructor Use Only.  
This document should not be distributed.

## Practice 11-1: Setting Up a Table with Sample Vectors

---

### Overview

In this practice, you will see how to perform the various Vector DDL operations.

### Assumption

You have completed Practices 1-1, 1-2, 1-3, and 10-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmr1p0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123###@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

Last Successful login time: Tue May 14 2024 20:46:30 +00:00

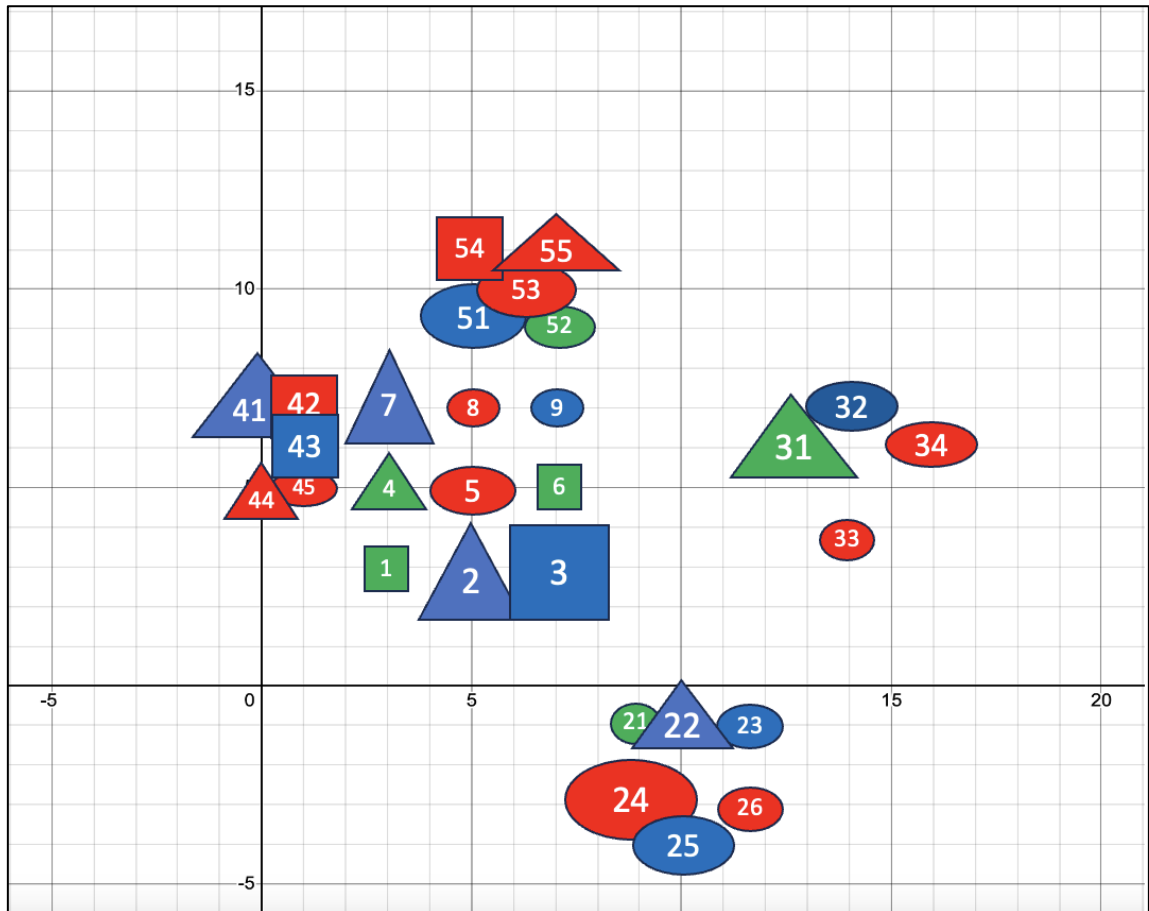
Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Set up a table with sample vectors

- a. You will create a table called `vt3` with vectors to map different shapes, colors, and sizes as illustrated in the figure below.



- b. Create a table to store the vectors.

```
SQL> DROP TABLE IF EXISTS vt3 ;

Table dropped.

SQL> CREATE TABLE vt3 AS SELECT * FROM vt2;

Table created.
```

```
SQL> SELECT * FROM vt3 ORDER BY 1;
```

| ID | V                    | VSIZE  | SHAPE    | COLOR |
|----|----------------------|--------|----------|-------|
| 1  | [3.0E+000,3.0E+000]  | Small  | Square   | Green |
| 2  | [5.0E+000,3.0E+000]  | Large  | Triangle | Blue  |
| 3  | [7.0E+000,3.0E+000]  | Large  | Square   | Blue  |
| 4  | [3.0E+000,5.0E+000]  | Small  | Triangle | Green |
| 5  | [5.0E+000,5.0E+000]  | Medium | Oval     | Red   |
| 6  | [7.0E+000,5.0E+000]  | Small  | Square   | Green |
| 7  | [3.0E+000,7.0E+000]  | Large  | Triangle | Blue  |
| 8  | [5.0E+000,7.0E+000]  | Small  | Oval     | Red   |
| 9  | [7.0E+000,7.0E+000]  | Small  | Oval     | Blue  |
| 21 | [9.0E+000,-1.0E+000] | Small  | Oval     | Green |
| 22 | [1.0E+001,-1.0E+000] | Medium | Triangle | Blue  |
| 23 | [1.1E+001,-1.0E+000] | Small  | Oval     | Blue  |
| 24 | [9.0E+000,-3.0E+000] | Large  | Oval     | Red   |
| 25 | [1.0E+001,-4.0E+000] | Medium | Oval     | Blue  |
| 26 | [1.2E+001,-3.0E+000] | Small  | Oval     | Red   |
| 31 | [1.3E+001,6.0E+000]  | Large  | Triangle | Green |
| 32 | [1.4E+001,7.0E+000]  | Medium | Oval     | Blue  |
| 33 | [1.4E+001,4.0E+000]  | Small  | Oval     | Red   |
| 34 | [1.6E+001,6.0E+000]  | Medium | Oval     | Red   |
| 41 | [0,7.0E+000]         | Large  | Triangle | Blue  |
| 42 | [1.0E+000,7.0E+000]  | Medium | Square   | Red   |
| 43 | [1.0E+000,6.0E+000]  | Medium | Square   | Blue  |
| 44 | [0,5.0E+000]         | Small  | Triangle | Red   |
| 45 | [1.0E+000,5.0E+000]  | Small  | Oval     | Red   |
| 51 | [5.0E+000,9.0E+000]  | Large  | Oval     | Blue  |
| 52 | [7.0E+000,9.0E+000]  | Small  | Oval     | Green |
| 53 | [6.0E+000,1.0E+001]  | Medium | Oval     | Red   |
| 54 | [5.0E+000,1.1E+001]  | Medium | Square   | Red   |
| 55 | [7.0E+000,1.1E+001]  | Medium | Triangle | Red   |

29 rows selected.

```
SQL>
```

4. Close the terminal window.

## Practice 11-2: Using Alternative Distance Functions

---

### Overview

In this practice, you use the following types of distance functions:

- Cosine Similarity
- Dot Product
- Manhattan Distance
- Hamming Distance

### Assumptions

You have completed Practice 11-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the vector user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Look for the four closest vectors to (16,4) using **Cosine Similarity** as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
FROM    vt3
ORDER BY vector_distance( vector('[16, 4]'), v, COSINE)
FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 3  | Large  | Square   | Blue  |
| 31 | Large  | Triangle | Green |

SQL>

4. Look for the four closest vectors to (16,4) using **Euclidean Similarity** as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
FROM    vt3
ORDER BY vector_distance( vector('[16, 4]'), v, EUCLIDEAN)
FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 31 | Large  | Triangle | Green |
| 32 | Medium | Oval     | Blue  |

SQL>

5. Look for the four closest vectors to (16,4) using **Dot Product** as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
FROM    vt3
ORDER BY vector_distance(vector('[16, 4]'), v, DOT)
FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 34 | Medium | Oval     | Red   |
| 32 | Medium | Oval     | Blue  |
| 33 | Small  | Oval     | Red   |
| 31 | Large  | Triangle | Green |

SQL>



6. Look for the four closest vectors to (16,4) using **Manhattan** as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
FROM   vt3
ORDER  BY vector_distance(vector('[16, 4]'), v, MANHATTAN)
FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 31 | Large  | Triangle | Green |
| 32 | Medium | Oval     | Blue  |

SQL>

7. Look for the four closest vectors to (16,4) using **Hamming** as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
FROM   vt3
ORDER  BY vector_distance( vector('[16, 4]'), v, HAMMING)
FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 1  | Small  | Square   | Green |
| 4  | Small  | Triangle | Green |

SQL>

8. Close the terminal window.

## Practice 11-3: Using Alternative Vector Distance Functions

---

### Overview

When using Euclidean as the distance function, there are four ways to find the closest vectors. In this practice, you will use the following vector distance functions.

- `L1_DISTANCE(v1, v2)` = MANHATTAN distance
- `L2_DISTANCE(v1, v2)` = EUCLIDEAN distance
- `COSINE_DISTANCE(v1, v2)` = COSINE similarity
- `INNER_PRODUCT(v1, v2)` = DOT product

### Assumptions

You have completed Practice 11-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrip0 ~]$ ssh-oracle
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1

SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19
2024
Version 23.4.0.24.05

Copyright (c) 1982, 2024, Oracle. All rights reserved.

Last Successful login time: Tue May 14 2024 20:46:30 +00:00

Connected to:
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -
Production
Version 23.4.0.24.05

SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Look for the four closest vectors to (16,4) using L1\_DISTANCE as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY L1_DISTANCE(vector('[16, 4]'), v)
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 31 | Large  | Triangle | Green |
| 32 | Medium | Oval     | Blue  |

SQL>

4. Look for the four closest vectors to (16,4) using L2\_DISTANCE as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY L2_DISTANCE(vector('[16, 4]'), v)
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 31 | Large  | Triangle | Green |
| 32 | Medium | Oval     | Blue  |

SQL>

5. Look for the four closest vectors to (16,4) using `COSINE_DISTANCE` as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY COSINE_DISTANCE(vector('[16, 4]'), v)
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 3  | Large  | Square   | Blue  |
| 31 | Large  | Triangle | Green |

SQL>

6. Look for the four closest vectors to (16,4) using `INNER_DISTANCE` as the vector distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY INNER_PRODUCT(vector('[16, 4]'), v)
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 44 | Small  | Triangle | Red   |
| 41 | Large  | Triangle | Blue  |
| 45 | Small  | Oval     | Red   |
| 43 | Medium | Square   | Blue  |

SQL>

7. Close the terminal window.

## Practice 11-4: Using Shorthand Syntax to Look for the Closest Vectors

---

### Overview

In this practice, you will use shorthand syntax to call a specific vector distance function.

The shorthand syntax includes the following functions:

- `v1 <-> v2` = Euclidean Distance
- `v1 <=> v2` = Cosine Similarity
- `v1 <#> v2` = Negative Dot Product

### Assumptions

You have completed Practice 11-1 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmrlp0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to ORCLPDB1 as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

3. Look for the four closest Vectors to (16,4) using <-> as the vector distance function.

**Note:** <-> is equivalent to the Euclidean Distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY vector('[16, 4]') <-> v
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 31 | Large  | Triangle | Green |
| 32 | Medium | Oval     | Blue  |

SQL>

4. Look for the four closest vectors to (16,4) using <=> as the vector distance function.

**Note:** <=> is equivalent to the vector Cosine Similarity distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY vector('[16, 4]') <=> v
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 33 | Small  | Oval     | Red   |
| 34 | Medium | Oval     | Red   |
| 3  | Large  | Square   | Blue  |
| 31 | Large  | Triangle | Green |

SQL>

5. Look for the four closest vectors to (16,4) using <#> as the vector distance function.

**Note:** <#> is equivalent to the vector Negative Dot Product distance function.

```
SQL> SELECT id, vsize, shape, color
      FROM vt3
      ORDER BY vector('[16, 4]') <#> v
      FETCH FIRST 4 ROWS ONLY;
```

| ID | VSIZE  | SHAPE    | COLOR |
|----|--------|----------|-------|
| 34 | Medium | Oval     | Red   |
| 32 | Medium | Oval     | Blue  |
| 33 | Small  | Oval     | Red   |
| 31 | Large  | Triangle | Green |

```
SQL>
```

6. Close the terminal window.

For Instructor Use Only.  
This document should not be distributed.



**Practices for Lesson 12:  
Testing Other Vector  
Functions**

For Instructor Use Only.  
This document should not be distributed.

## Practice 12-1: Other Vector Functions

---

### Overview

In this practice, you will see the following vector operations:

- Functions for operating on vectors
- Vector conversion and binding operations

### Assumptions

You have completed Practices 1-1, 1-2, and 1-3 successfully.

### Tasks

1. Open a terminal window and connect to the OCI DB system using the `ssh-oracle` alias.

```
[oracle@edvmr1p0 ~]$ ssh-oracle  
[oracle@host01 ~]$
```

2. Using SQL\*Plus, connect to `ORCLPDB1` as the `vector` user.

```
[oracle@host01~]$ sqlplus vector/WEIcome123##@orclpdb1  
  
SQL*Plus: Release 23.0.0.0.0 - Production on Tue May 14 20:49:19  
2024  
Version 23.4.0.24.05  
  
Copyright (c) 1982, 2024, Oracle. All rights reserved.  
  
Last Successful login time: Tue May 14 2024 20:46:30 +00:00  
  
Connected to:  
Oracle Database 23ai Standard Edition Release 23.0.0.0.0 -  
Production  
Version 23.4.0.24.05  
  
SQL>
```

For Instructor Use Only.  
This document should not be distributed.

3. Use functions for operating on vectors.

- a. Convert a vector from a string. **Note:** The `to_vector()` function creates a vector from a string. It is equivalent to the `vector()` constructor.

```
SQL> SELECT to_vector('[34.6, 77.8]', 2, float32) FROM dual;

TO_VECTOR(' [34.6,77.8] ',2,FLOAT32)
-----
[3.45999985E+001,7.78000031E+001]

SQL> SELECT to_vector('[34.6, 77.8, -89.34]', 3, float32);

TO_VECTOR(' [34.6,77.8,-89.34] ',3,FLOAT32)
-----
[3.45999985E+001,7.78000031E+001,-8.93399963E+001]

SQL>
```

- b. Return the norm value of a vector. **Note:** It is the distance from the origin to the vector.

```
SQL> SELECT vector_norm(vector('[4, 3]', 2, float32) );

VECTOR_NORM(VECTOR(' [4,3] ',2,FLOAT32))
-----
5.0E+000

SQL> SELECT vector_norm(vector('[4, 3]', 2, float64) );

VECTOR_NORM(VECTOR(' [4,3] ',2,FLOAT64))
-----
5.0E+000

SQL> SELECT vector_norm(vector('[4, 3]', 2, int8) );

VECTOR_NORM(VECTOR(' [4,3] ',2,INT8))
-----
5.0E+000

SQL>
```

- c. Determine the number of dimensions in a vector. `vector_dimension_count()` returns the number of dimensions in a vector. **Note:** In this example the third select statement is expected to return an error as there are four dimensions in the vector but the calling function specifies that there are only three.

```
SQL> SELECT vector_dimension_count(vector('[34.6, 77.8]', 2,
float64));

VECTOR_DIMENSION_COUNT(VECTOR('[34.6,77.8]',2,FLOAT64))
-----
                                                                2

SQL> SELECT vector_dimension_count(vector('[34.6, 77.8, 9]', 3,
float32));

VECTOR_DIMENSION_COUNT(VECTOR('[34.6,77.8,9]',3,FLOAT32))
-----
                                                                3

SQL> SELECT vector_dimension_count(vector('[34.6, 77.8, 9, 10]',
3, int8));

SELECT vector_dimension_count(vector('[34.6, 77.8, 9, 10]', 3,
int8))
*

ERROR at line 1:
ORA-51803: Vector dimension count must match the dimension count
specified in
the column definition (actual: 4, required: 3).
Help: https://docs.oracle.com/error-help/db/ora-51803/

SQL>
```

- d. Determine the format of the vector numbers. `vector_dimension_format()` returns the format of the vector numbers. **Note:** In this example the third select statement is expected to return an error as there are four dimensions in the vector but the calling function specifies that there are only three.

```
SQL> SELECT vector_dimension_format(vector('[34.6, 77.8]', 2,
float64));

VECTOR_DIMENSION_FORMAT(VECTOR('[34.6,77.8]',2,FLO
-----
FLOAT64

SQL> SELECT vector_dimension_format(vector('[34.6, 77.8, 9]', 3,
float32));

VECTOR_DIMENSION_FORMAT(VECTOR('[34.6,77.8,9]',3,F
-----
FLOAT32

SQL> SELECT vector_dimension_format(vector('[34.6, 77.8, 9,
10]', 3, int8));

SELECT vector_dimension_format(vector('[34.6, 77.8, 9, 10]', 3,
int8))

*

ERROR at line 1:
ORA-51803: Vector dimension count must match the dimension count
specified in
the column definition (actual: 4, required: 3).
Help: https://docs.oracle.com/error-help/db/ora-51803/

SQL>
```

4. Convert a vector to a string or CLOB.

- a. Convert a vector to a string or CLOB using `vector_serialize()`.

```
SQL> SELECT vector_serialize(vector('[1.1, 2.2, 3.3]', 3,
float32));

VECTOR_SERIALIZE(VECTOR('[1.1,2.2,3.3]',3,FLOAT32))
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]

SQL> SELECT vector_serialize(vector('[1.1, 2.2, 3.3]', 3,
float32) returning varchar2(1000));

VECTOR_SERIALIZE(VECTOR('[1.1,2.2,3.3]',3,FLOAT32)RETURNINGVARCH
AR2(1000))
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]

SQL> SELECT vector_serialize(vector('[1.1, 2.2, 3.3]', 3,
float32) returning clob);

VECTOR_SERIALIZE(VECTOR('[1.1,2.2,3.3]',3,FLOAT32)RETURNINGCLOB)
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]

SQL>
```

- b. Convert a vector to a string or CLOB using `from_vector()`. **Note:** `from_vector()` converts a vector into a string or a clob. It is equivalent to the `vector_serialize()` function.

```
SQL> SELECT from_vector(vector('[1.1, 2.2, 3.3]', 3, float32));

FROM_VECTOR(VECTOR('[1.1,2.2,3.3]',3,FLOAT32))
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]

SQL> SELECT from_vector(vector('[1.1, 2.2, 3.3]', 3, float32)
returning varchar2(1000));

FROM_VECTOR(VECTOR('[1.1,2.2,3.3]',3,FLOAT32)RETURNINGVARCHAR2(1
000))
-----
[1.10000002E+000,2.20000005E+000,3.29999995E+000]
```

```
SQL> SELECT from_vector(vector('[1.1, 2.2, 3.3]', 3, float32)
returning clob);
```

```
FROM_VECTOR(VECTOR('[1.1,2.2,3.3]',3,FLOAT32)RETURNINGCLOB)
```

```
-----  
[1.10000002E+000,2.20000005E+000,3.29999995E+000]
```

```
SQL>
```

5. Close the terminal window.

For Instructor Use Only.  
This document should not be distributed.